Solution Design Document

SIMF, Saudi International Maritime Forum

Low-Level Design, SIMF-LLD-003, Version 1.1

DOCUMENT INFORMATION

Sr. No.	Date	Ver.	Name	Action	Description
1	2026-07-21	1.0	SIMF Engineering & Architecture Team	Create	Initial issue of the Solution Design Document (Low-Level Design) authored to the DoD LLD template, sourced from the SIMF design baseline and the source tree.
2	2026-08-10	1.1	SIMF Engineering & Architecture Team	Update	Added the component, data flow and deployment figures. Corrected the real-time transport statements after SIMF.RealTime was removed from the solution.

Overview

This Solution Design Document (SDD) is the detailed technical design for the SIMF platform. It follows the DoD Solution Design Document structure and is grounded in the SIMF controlled documents and the source tree: it is a reference baseline for development, review and QA. Where a section catalogues a large set (use cases, screens, tables), it names the authoritative source for the exhaustive per-item detail rather than duplicating it.

Contents

Right-click and “Update Field” to build the table of contents.

1. Introduction

1.1 Purpose

This Solution Design Document (SDD / Low-Level Design), SIMF-LLD-003, defines the technical solution design for the SIMF, Saudi International Maritime Forum platform: how the system is structured, the classes and interfaces that implement each concern, the request pipeline, the database schema, the client applications, the shared libraries, and the runtime flows that connect them. It is written to the DoD "Solution Design Document" template structure (seven sections; section 7 = Solution Description).

The intended audience is the engineering team implementing and maintaining SIMF (architects and developers), the QA function verifying it, and the Ministry of Defense reviewer assessing the design. The document is the reference baseline against which development is carried out and against which the delivered system is reviewed, it is detailed enough to implement against and to review against.

This SDD describes the objective of the document itself, not a re-statement of business requirements; the functional requirements and roles/permissions catalogue live in their own controlled documents (see section 1.4). Where a needed low-level detail is not present in the SIMF source documents or source tree, this document marks it explicitly rather than inventing it.

1.2 Scope

This design covers the whole SIMF platform, consistent with the component set named for the SIMF architecture:

- **Backend API**: `SIMF.Api`, a .NET 10 / FastEndpoints HTTP host with route prefix `api/v1`, layered as Domain, Application, Infrastructure, Api.
- **Control Panel (CP)**: a Blazor Server administrative application over the `/admin/*` surface, with per-page/per-action permission gating.
- **Website**: a public Blazor SSR site with interactive islands for the auth and account flows.
- **Mobile application**: a Flutter app (Android + iOS) with a persistent 5-tab shell, built on Riverpod, go_router and Dio.
- **Two databases**: the physically separated `SIMF_Identity` (`SimfIdentityDbContext`) and `SIMF_App` (`SimfAppDbContext`) SQL Server databases, with separate migration histories.
- **Shared libraries**: `SIMF.Common`, `SIMF.Contracts`, `SIMF.ApiClient`, `SIMF.Components`.

The feature/module surface covered is the full bounded-context set: Auth/IdentityAccess, Account/MyArea, Programme (sessions/themes/speakers/presentations/summaries/recordings), SeatReservations/Bookings, Gates, Exhibition/Exhibitors/Booths, Sponsors, News/Media, Archive, Notifications, Networking, MeetingRequests + BusinessMeetings, Statistics, Configuration, Organisations, Contacts, FAQ, Feedback, CMS, AI, Venue, Operations, and Auditing/Logs. Later sections cover every context; where full prose is impractical they use index tables and point to the authoritative source for exhaustive per-item detail rather than dropping items.

1.2.1 Out of scope

The following are intentionally excluded from this design, drawn from the open items and deferrals recorded in the sources:

- **Real-time push. Live notifications and Q&A are delivered over REST, polled by the client on a bounded interval. No server-push transport ships in this build.**
- **Deferred scale-out / distributed cache.** Distributed Redis caching (`IDistributedCache`) is not part of the current design; the template's Redis packages are conditional ("include if Redis is used"). No scale-out cluster topology is specified in the SIMF sources.
- Load-test thresholds and the monitoring/alerting toolchain, unset in the source.
- **Geofence attendance chain and a production live-video provider integration** beyond the current YouTube-iframe approach: the geofence arrival/attendance chain is a deferred open item, and live video uses YouTube with HLS/MP4 fallback as the current design.
- **Owner/operations security actions**: the mobile self-signed-TLS bypass (development/PoC) and committed development secrets are flagged for owner/operations remediation before handover, not code deliverables of this design.
- **External systems themselves**: SMTP server, the pluggable AI provider back-ends, and YouTube are integration endpoints, not components designed here.

1.3 Definitions and Acronyms

Built from the SIMF architecture baseline, extended with SIMF-specific terms verified against the source.

Term / Acronym	Definition
API	Application Programming Interface, the FastEndpoints HTTP backend (`SIMF.Api`)
CP	Control Panel, the Blazor Server admin application
BFF	Backend-for-Frontend, the thin server-side endpoint layer in CP/Website that holds tokens so the browser never does
DDD	Domain-Driven Design
JWT	JSON Web Token, HS256 access token issued by the token service
TOTP	Time-based One-Time Password, authenticator-app second factor (admins)
OTP	One-Time Password, e-mailed code second factor (visitors)
RBAC	Role-Based Access Control
PII	Personally Identifiable Information, encrypted at rest via `IPiiEncryptor`
RTL / LTR	Right-to-Left / Left-to-Right text direction (Arabic / English)
NCA	National Cybersecurity Authority (Saudi Arabia)
SSR	Server-Side Rendering (Blazor, Website)
SoT	Source of Truth
ApiResult / ApiError	The standard response envelope

	(`ApiResult<T>`) and its error object
PermissionCatalog	Single source of truth for permission codes (`Page.Action`), reused by API and CP
GridQuery / GridPage	Admin-grid paging request/response contract in `SIMF.Common`
MobileAppRole	The mobile privilege model: None / Visitor / Staff / Moderator
EF Core	Entity Framework Core, the code-first ORM
Soft delete	`IsActive`-based deactivation via `Deactivate()`; list queries filter `IsActive == true`
Logical FK	A cross-database user reference stored as a bare `Guid`, resolved on read, never a DB constraint
RSNF	Royal Saudi Naval Forces, the forum's host
SIMF	Saudi International Maritime Forum, the platform this document designs

1.4 References

The SIMF controlled documents and their governing area:

1. Software Engineering Standards (structure, DDD layering, conventions, naming, source control, code review, testing, security baseline, Definition of Done, freeze).

2. API Specification (the `ApiResult<T>` envelope, standard headers, error model, HTTP status codes, pagination, authentication endpoints; authoritative endpoint catalogue).

3. Software Architecture Document (modular monolith, bounded contexts, security, integration, deployment).

4. Mobile Application Architecture (Flutter app, Android + iOS).

5. Documentation Management Plan (documentation management).

6. Data Model and Database Design: the logical data model by bounded context, conventions, and the core ERD.

7. Roles and Permissions: the page-and-action permission catalogue, and the account-state and booking-status enums.

8. Control Panel Design.

9. Deployment and Operations: environments, CI/CD, health, rollback, and observability.

10. High-Level Design: the architecture document that serves as the entry point above this solution design document.

11. Low-Level Design: the detailed component and data design that is the primary source for this document's low-level content.

12. Software Requirements Specification.

13. The Functional Design Specification set, one per module (001 Authentication & Login; 002 Registration & Approval; 003 Badge & Access Control; 004 Forum Programme; 005 Bookings & Attendance; 006 Exhibition; 007 Engagement; 008 Networking & Cognitive AI; 009 Notifications; 010 Media, News & Archive; 011 Statistics & Dashboards; 012 Control Panel Configuration; 013 Business Meetings; 014 Contacts & Sharing): the authoritative per-module functional detail referenced throughout section 5.

14. Business Requirements Document: the business objectives, scope, stakeholders and business requirements, issued in Arabic and English editions.

2. Process View

This section describes how SIMF behaves at runtime: how the deployed components communicate, which pipeline every request passes through, and how the highest-value business flows execute end to end. The static structure (projects, layering, schema) is covered in later sections; here the concern is dynamic behaviour. Where a capability is planned rather than built, that is stated rather than assumed.

2.1 Process Interaction

2.1.1 Participating components

At runtime SIMF is a small set of long-running processes talking over HTTP, plus a database and in-process background workers. There is one API and it is the only writer to the databases; the three client surfaces are all clients of that one API.

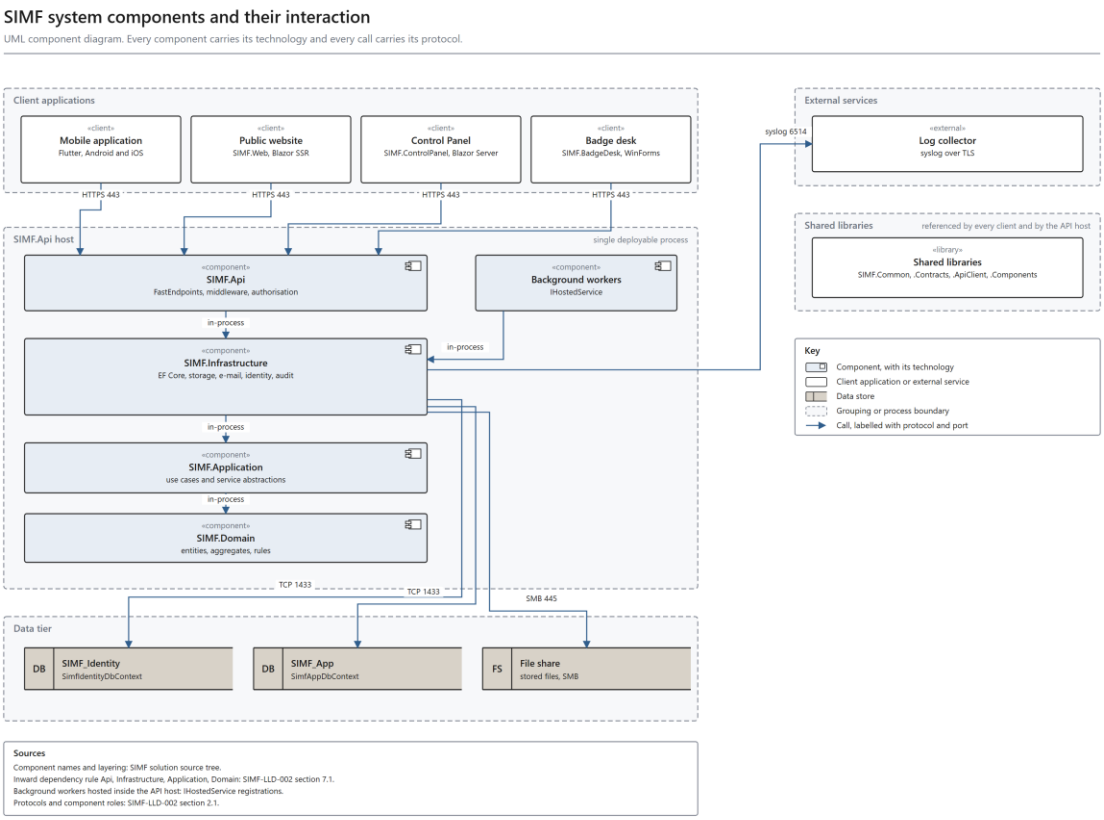

Figure 1. SIMF system components and their interaction. A UML component diagram: every component with its technology, and every call labelled with its protocol and port.

Component	Kind	Talks to	Protocol / mechanism
Mobile app (Flutter)	Client	API	REST over HTTPS
Control Panel (Blazor Server)	Client	API	REST via typed client
Website (Blazor SSR + islands)	Client	API	REST via typed client
SIMF.Api (FastEndpoints host)	Server	Both DBs, e-mail	EF Core; SMTP
SimfIdentityDbContext / SimfAppDbContext	Persistence	-	EF Core to two SQL Server databases
Background workers	In-process hosted services	App DB, e-mail	EF Core; SMTP

2.1.2 Client-to-API communication

- **All three clients call the same REST API** under route prefix `api/v1`, split by convention into an app surface (`/api/v1/app/*`) and an admin surface (`/api/v1/admin/*`), exposed as two OpenAPI documents.
- **Every response uses the `ApiResult<T>` envelope**: `Success`, `Data`, `Error`, `Meta`, so clients branch on a single predictable shape; `ApiCallResult<T>` pairs the envelope with the HTTP status so a client can react to 401/403.

- **Mobile** uses a single `SimfApiClient` (Dio) with interceptors that attach `X-App-Key`, `X-Device-Type`, `Accept-Language` and `Authorization: Bearer <token>`; on a 401 it refreshes once via `POST /app/auth/refresh` and replays, with concurrent 401s sharing one in-flight refresh (single-flight).
- **Control Panel and Website** are server-side Blazor apps that reach the API through the typed `SIMF.ApiClient` (`SimfAuthClient`, `SimfAccountClient`, `SimfAdminClient`, `SimfPublicClient`). This is a server-to-server call: the browser never holds a raw bearer token. Under the BFF pattern the CP/Website endpoint layer performs the credential and second-factor steps, stores tokens in an encrypted cookie, holds the access token per-circuit in `SimfAuthSession`, and forwards it to the API on each call. Network faults are mapped to a failed `ApiResult` rather than thrown.

2.1.3 API-to-database and background communication

- The API owns two physically separated databases through two EF Core contexts: `SimfIdentityDbContext` (users, roles, permissions, tokens) and `SimfAppDbContext` (all business data). Cross-database references are bare `Guid` "logical FKs" resolved on read, never a cross-DB join.
- **Background workers** run in-process as hosted services and do not participate in the request path: `DormantAccountSweepService`, `RegistrationGateAutoCloseWorker`, `SessionReminderWorker`, and `EmailBackgroundService` (drains an `EmailQueue` and sends via MailKit SMTP). Notification and e-mail sending is therefore asynchronous: a request queues work and a worker drains it.

Real-time note (as-built): live updates are delivered over REST, polled by the client on a bounded interval. This section describes that REST behaviour. No server-push transport ships in this build.

2.1.4 The request/middleware pipeline

Every API request passes through a fixed middleware pipeline before reaching an endpoint. Host configuration also applies JWT bearer plus a distinct `StreamToken` scheme, named and dynamic permission authorization policies, an explicit CORS allow-list, per-IP and named rate limiting, and `GET /health`.

Middleware pipeline (classes and responsibilities):

Class	Responsibility
CorrelationIdMiddleware	Read/generate `X-Correlation-Id`; enrich logs
SecurityHeadersMiddleware	Apply baseline security response headers
ErrorHandlingMiddleware	Convert exceptions to `ApiResult.Fail` with the correct status
EmailRateLimitKeyMiddleware	Extract e-mail from credential bodies for per-e-mail throttling
SwaggerBasicAuthMiddleware	Gate the OpenAPI UI in production

2.1.5 Authentication and permission flow

- **Endpoints declare their own authorisation** in `Configure()`: `AllowAnonymous()` for sign-in/sign-up/password-reset only, or `Policies(...)` naming a permission policy plus `RequireApprovedAccount` for admin actions.
- **Tokens** are HS256 JWTs issued by `JwtTokenService`, carrying `sub`, `email`, `jti`, `security_stamp`, `account_state`, `user_type`, `mobile_app_role`, one claim per role, and one `perm` claim per permission code (the Administrator wildcard collapses to a single `*`). JWT validation pins the algorithm to HS256, validates issuer/audience/lifetime/signing, and runs a security-stamp check so revoking a role or disabling an account takes effect immediately.
- **Permission enforcement** uses `PermissionCatalog` as the single source of truth. `PermissionPolicyProvider` materialises `perm:<code>` policies on demand and `PermissionAuthorizationHandler` passes if the caller's `perm` claims contain the code or the wildcard. The same catalogue is reused by the Control Panel, so API and CP enforce identical codes.

2.2 Process Flow

The four flows below are the primary end-to-end business processes. Each is a textual walkthrough with numbered steps, decision points and exception paths. These flows mirror the sequence diagrams described elsewhere in this document; refer there for the diagram form.

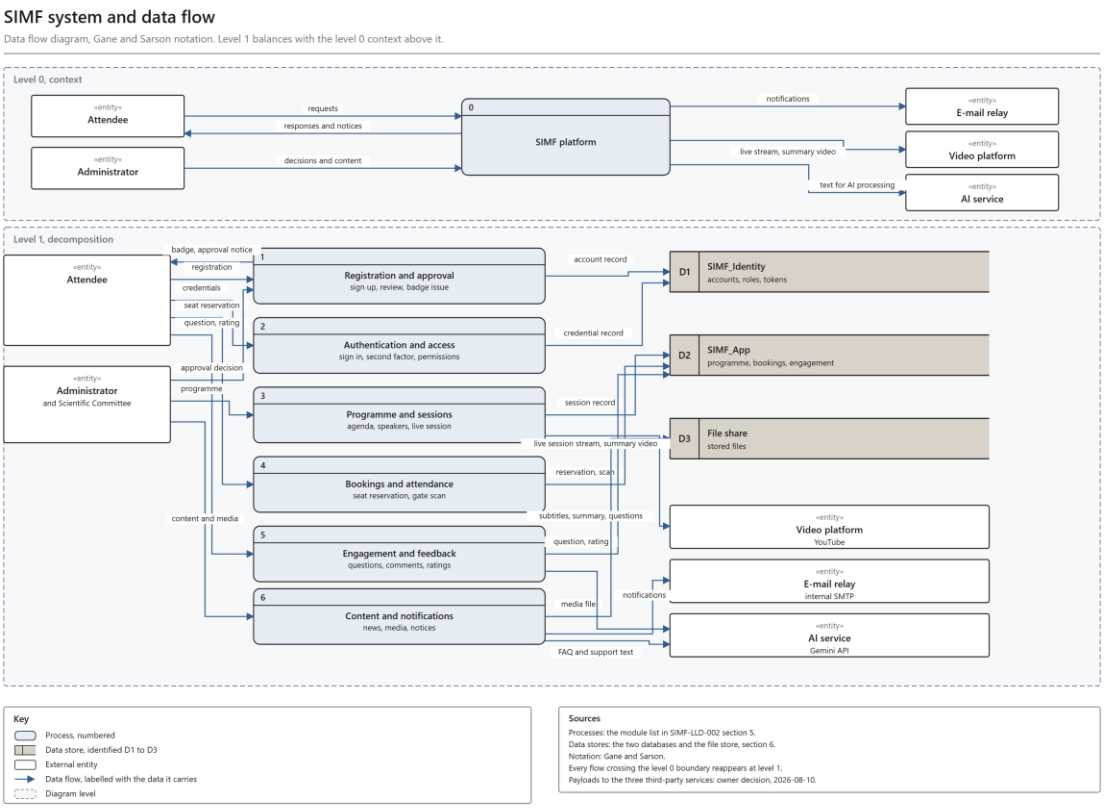

Figure 2. SIMF system and data flow. A data flow diagram in Gane and Sarson notation: a level 0 context view and a level 1 decomposition that balances with it. The four walkthroughs below trace paths through it.

2.2.1 Visitor sign-in with e-mail OTP

1. The visitor submits e-mail and password to the app; the app calls `POST /app/auth/sign-in`. This route is anonymous and rate-limited under the `auth` / `auth-email` policies.

2. The API verifies the credential. Decision point: credential valid? If not, it returns an `ApiResult.Fail` (invalid-credential) and the flow stops (exception path).

3. On valid credentials the API issues a one-time code (`AccountCode`, purpose `SignInOtp`) and queues an e-mail; `EmailBackgroundService` drains the queue and sends via SMTP. The API responds that a second factor is required.

4. The visitor enters the code; the app calls `POST /app/auth/verify-otp`.

5. Decision point: code correct and unexpired? The API checks the code against `ExpiresAt`/`ConsumedAt`/`AttemptCount`. If wrong or expired, it returns a failure and increments the attempt count (exception path; repeated failures are throttled).

6. On success the API issues an HS256 access token (with `perm`/role claims) and a hashed, rotating refresh token; the session carries an absolute 24-hour cap.

7. The app stores the tokens and the `HeadersInterceptor` attaches the bearer on subsequent calls; a later 401 triggers a single-flight refresh via `POST /app/auth/refresh`.

Admin variant: administrators use TOTP (with single-use recovery-code fallback) rather than e-mailed OTP.

2.2.2 Admin permission-gated action

1. An admin triggers an action in the Control Panel (e.g. list visitors). The CP calls the API server-to-server via `SimfAdminClient`, forwarding the per-circuit access token.

2. The request passes the middleware pipeline (correlation id, security headers, error handling) and reaches JWT authentication.

3. Decision point: token valid and security-stamp current? If the token is invalid, expired, or the security stamp no longer matches (role revoked / account disabled), authentication fails and the API returns 401 (exception path).

4. The endpoint declares `Policies(PermissionCatalog.PolicyFor(<code>), RequireApprovedAccount)`. `PermissionPolicyProvider` materialises the `perm:<code>` policy and `PermissionAuthorizationHandler` checks the caller's `perm` claims.

5. Decision point: caller holds the code or `*` and is approved? If not, the API returns 403 and the CP surfaces the gate (exception path). The CP itself also gates the page (`[RequirePermission]`), the nav item and the action button, so the gate is enforced on both sides.

6. On pass, `HandleAsync()` runs the use case, and mutating operations are stamped and row-audited by the SaveChanges interceptors; the response returns as `ApiResult<T>`.

2.2.3 Seat reservation (auto-confirm, check-in confirmed)

1. An approved visitor calls `POST /app/sessions/{sessionId}/seats/reserve`; the endpoint validates the request and resolves the caller from the JWT under `RequireApprovedAccount`.

2. The seat-reservation service loads the `Session` and its `Hall`/`HallSeatLayout` and reads the `SeatSelectionMode`.

3. Decision point, is the seat free? Freeness is enforced by a filtered unique index (one active reservation per seat per session). If the seat is taken, the write fails and the API returns a conflict (exception path).

4. The service creates a `SeatReservation` (`Kind = UserBooking`; `Status = Approved` on create, reservation-only auto-confirm) and saves; the audit interceptors stamp and row-audit the insert.

5. The reservation is confirmed immediately and stands as a provisional hold; a pre-start sweep auto-releases any hold not checked in shortly before the session starts.

6. The attendee checks in at the hall gate (staff scans their QR), which marks the held seat confirmed (checked-in). (The legacy approve/reject endpoints are retained but dormant.)

- Gate check-in confirms the seat (checked-in); no admin approval transition and no booking notification are raised on the reservation path.
- Reject to `Status = Rejected` with a reason, and a `BookingRejected` notification is queued (exception/alternate path).

7. The app shows an inline success message on reservation; no booking notification is queued or delivered.

2.2.4 Gate scan, idempotent

1. A staff member scans an attendee badge QR at a gate; the app calls `POST /app/gates/{gateId}/scans`, gated by the staff `mobile_app_role` and the gate permission policy.

2. Decision point, duplicate scan? The service consults `ScanIdempotency`; if this scan has already been recorded, it returns the prior outcome without writing a second row (idempotent path). This is the core guarantee of the flow.

3. On a first-seen scan the service resolves the attendee and the gate's allowed profile types (`GateProfileTypeAllow`) and evaluates the scan.

4. Decision point, is entry permitted for this attendee at this gate? The outcome is captured as a `GateScan` row, an append-only table (bigint identity) that stores actor/subject snapshot columns and is excluded from row-audit because it is itself an immutable log.

5. The API returns the scan outcome (allowed / denied, with reason) as `ApiResult<T>`; the staff app shows the result. Denials and errors are outcomes on the same append-only record, not exceptions that lose the scan (see `ScanOutcome`).

3. System Design

This section presents the SIMF use-case model: the primary actors, a system-boundary summary of the major use cases across every module, and detailed use-case realisations for five key modules with a compact per-module index for the remainder. It is grounded in the Feature Design Specification set, the low-level design, and the page inventory. Use-case identifiers (UC-nn) are those declared for each feature's "Requirements and use cases covered" table; where a shipped feature has no numbered use case, the row cites its governing feature specification instead of inventing a UC number.

3.1 Use Case Realization

3.1.1 High Level Use Case

Primary actors

The SIMF actor set is drawn from the account/role model (including `MobileAppRole`) and the role/permission model those docs reference. The mobile privilege enum is `Guest / Visitor / Staff / Moderator`; Control Panel access is roles-only with `Administrator = "*"`.

Actor	Surface	Description
Anonymous / Public	Website, App (Guest)	Unauthenticated visitor browsing public content (programme, speakers, news, archive, delegations, venue map). No account.
Visitor / Attendee	App, Website	Self-registered, approved end user (`UserType = Visitor`, `MobileAppRole = Visitor`), books seats, asks questions, rates, networks.
VIP / Delegate	App, CP-managed	Visitor flagged VIP/VVIP or `IsDelegate` with an invited country; CP-registered welcome roster (routes `/admin/visitors/vip`, `/admin/delegates`; delegations flag).
Moderator	App (`MobileAppRole = Moderator`)	Runs a per-session question desk for assigned sessions only.
Gate operator / Staff	App (`MobileAppRole = Staff`)	Scans badges at gates/hall doors, captures exhibitor leads, runs on-site registration desk.
Administrator / Organiser	Control Panel	`UserType = Admin`; every CP action gated by a permission code; TOTP second factor.
Scientific committee	Control Panel	Role-holder who reviews the programme, live broadcast, and the Q&A "team" stage.
Public Relations (PR)	Control Panel	Role-holder who approves exhibitors and assigns booths (`AppRoles.PublicRelations`). The legacy booking-approval queue is retained but dormant.
System / background workers	Server	Dormant-account sweep, registration-gate auto-close, session reminders, rating prompts, e-mail dispatch.

Note: the audience gate historically used "has any RBAC role = staff"; the `UserType`/`ProfileType` model is the authoritative target and is reflected as built. Reviewer sub-roles (Scientific committee, PR) are the "suggested configuration" role assignments, not fixed system roles.

System-boundary use-case summary

Grouped by module. One row per major use case. "Actor" lists the primary actor(s); UC ids are used where the governing specification names one.

UC id	Name	Actor(s)	Description
Authentication			
UC-01	Create an account	Anonymous	Sign up with e-mail + password to `Registered`.
UC-02	Verify the e-mail	Anonymous	Submit 6-digit code to `EmailVerified`; resend supported.
UC-04	Sign in	Visitor, Admin	Password + account-state branch; email-OTP (visitor) / TOTP (admin) second factor; audience gate cp/web/app.
UC-06	Reset a password	Any	Forgot/reset via e-mailed code; invalidates active sessions.
-	Refresh / sign-out	Any signed-in	Rotate refresh token (24 h cap); revoke on sign-out.
-	Enrol / reset TOTP	Admin	Two-slot authenticator enrolment + 10 recovery codes; admin-driven 2FA reset.
Registration & Approval			
UC-03	Complete registration	Visitor, Exhibitor	Profile + identity + attachments + venue track + terms to `PendingApproval`.
UC-07	View registration status	Visitor	Track stages: data sent, then email, then security review, then activation.
UC-20	Review & decide a registration	Admin, Scientific	Approve (set final type, mint QR id, issue badge) or reject with reason; bulk approve.
UC-21	Approve an exhibitor	PR	One-stage approve plus assign booth.
UC-31	Open / close registration	Admin	Manual plus auto-close at last forum day.
UC-34	Register on-site / reprint badge	Staff	Desk search then reprint or on-site register.
Programme			
UC-23	Manage sessions	Scientific, Admin	CRUD sessions, link theme/hall/speakers, live flag.
UC-24	Manage halls & seating	Admin	Halls, capacity, seat grid/layouts.
UC-25	Manage speakers	Admin	Speaker profiles plus presentations.
UC-08	Browse the agenda &	Public, Visitor	Day filter, search, session

	a session		detail, add-to-calendar, reminders.
Bookings and Attendance			
UC-09	Book a session seat	Visitor	Pick seat, no-overlap rule, reserve seat then confirmed immediately (provisional hold until gate check-in).
UC-10	Cancel a booking	Visitor	Cancel before session start; seat released.
UC-22	Approve a booking (legacy, retained but dormant; no attendee booking enters Pending)	PR	Legacy approve/reject/bulk-approve retained but dormant; attendee reservations auto-confirm on create and confirm at gate check-in.
	View seat map / My Seat	Visitor	Available / taken / mine; assigned seat on detail.
Gates & Access			
UC-11	View my badge & QR	Visitor	Badge card + signed-token QR.
UC-33	Verify a badge at venue entry	Staff	Scan then verify then record `GateScan`/`VenueEntry`.
UC-35	Check in at a hall door	Staff, device	QR scan + geofence to `HallAttendance`.
UC-12	Scan another attendee's badge	Visitor	Save a contact from a badge QR.
Exhibition			
UC-18	Browse exhibitors, booths & venue map	Public, Visitor	Booth directory, sponsors by tier, 2D venue map + navigate.
	Capture exhibitor lead	Staff (Exhibitor)	Scan visitor badge to "my visitors".
Sponsors			
UC-18	View sponsors	Public, Visitor	Sponsor list/detail by tier.
News / Media / Archive			
UC-32	Manage & view media, news, archive	Admin (manage); Public (read)	Media Center, news and categories, gallery, media partners, previous editions and visibility.
Engagement / Q&A			
UC-13	Watch a live session	Visitor	Live stream, captions and language, and an optional notification to users near the venue.

UC-14	Ask a question in a session	Visitor	Recipient speaker/host; arrival-gated; Pre/Live phase.
UC-36	Manage questions of an assigned session	Moderator	Order/hide/on-air/answer.
UC-15	Comment on a session	Visitor	Post comment.
UC-26	Moderate comments	Admin, Scientific	AI filter then admin approve/discard.
	Rate a session / day / event	Visitor	Multi-trigger ratings (day, programme-end, session-end, live-close).
Networking			
UC-16	Request a one-to-one meeting	Visitor	Meeting request then approval.
UC-17	Use the AI assistant	Visitor	Assistant over two-level FAQ.
	Meet people like you	Visitor	Interest-based matchmaking and 80%-or-higher recommendation/push.
	AI session summary	Visitor	Generated per-session summary.
	Accessibility AI	Visitor	Sign-language, speech, captions.
Meetings, B2B/B2C			
	Schedule a business meeting	Admin	Configure hall/tables, reserve over a slot, schedule 2+ parties to Confirmed (see section 5).
	Request a meeting-in-hall	Visitor	Attendee request to admin review against hall availability to speaker double-opt-in (build-ready).
Notifications			
UC-19	Receive a notification	Any signed-in	One abstraction, channels in-app/email/SMS/WhatsApp; inbox; reminders (see section 5).
Statistics			
UC-30	View the statistics dashboard	Admin	Per-day, overall, live attendance, GPS presence (see section 5).
Configuration			
UC-28	Manage dynamic content & categories	Admin	Content blocks, banners, categories, labels, colours, venue tracks (see section

			5).
UC-31	Open / close registration	Admin	Registration control (see section 5).
	View the operation log	Admin	History of every change.
Contacts & Sharing			
	Share my contact (QR / vCard)	Visitor	Consent-by-action share token plus vCard (Track 2).
	Scan / save a contact	Visitor	Resolve a shared token into My Contacts (Track 2).

This summary is representative of the major use cases per module; the exhaustive per-page and per-action inventory covers the Control Panel, Website, and roughly 34 mobile screens, together with the per-page E2E catalogue. No module is omitted.

3.1.2 Low Level Use Cases

The five key modules below carry full use-case tables grounded in their FDS. Flows are numbered to the FDS processing steps; error/exception codes are the FDS's own.

Module 1, Authentication / Login

UC-04, Sign in (with second factor)

Field	Value
Use Case ID	UC-04
Name	Sign in
Actor	Visitor (web/app), Administrator (CP)
Preconditions	Account exists and is `EmailVerified` or beyond; caller supplies the surface audience (cp/web/app).
Main Flow	1. Actor submits e-mail and password. 2. System finds account by e-mail and verifies password hash. 3. System branches on account state. 4. If second factor applies, TOTP for a user with an authenticator key, else visitor email-OTP, return `mfaRequired` plus a short-lived token. 5. Actor submits the TOTP/OTP code; system validates it and the mfa/otp token. 6. Audience gate: user's surface must match (cp to Admin; web/app to Visitor). 7. Issue access token (5 min) plus rotating refresh token (24 h cap). 8. Write `SignIn.Succeeded` to the operation log.
Alternative Flows	A1 `TwoFactorEnabled = false`: tokens issued directly on the password step, `MfaRequired = false`. A2 `PendingApproval` on web/app: allowed with a limited "status view" session; on cp: refused. A3 Recovery-code path replaces TOTP via `/auth/verify-recovery-code`.
Exception Flows	Bad credentials to `AUTH_INVALID_CREDENTIALS` (generic, 401).

	Unverified to `AUTH_EMAIL_NOT_VERIFIED` (403). Rejected/Disabled/Locked to `AUTH_ACCOUNT_NOT_APPROVED` / `AUTH_ACCOUNT_DISABLED` / `AUTH_ACCOUNT_LOCKED` (403). Wrong second factor to `AUTH_TOTP_INVALID` / expired mfa token `AUTH_MFA_TOKEN_EXPIRED` (400). Wrong surface to 403 `AUTH_WRONG_SURFACE_CP` / `AUTH_WRONG_SURFACE_WEB`. Rate exceeded to `RATE_LIMIT_EXCEEDED` (429).
Postconditions	A session (token pair) exists; access token carries account state, user type, permissions, `mobile_app_role`; audit rows written.

Companion authentication use cases (same module): UC-01 Sign-up, UC-02 Verify email plus resend, UC-06 Password reset, token refresh/sign-out, TOTP enrol/recovery/admin-reset. The full endpoint surface is documented alongside this module.

Module 2, Registration & Approval

UC-03 / UC-20, Complete registration and decide it

Field	Value
Use Case ID	UC-03 (register), UC-20 (review & decide)
Name	Complete registration profile; organiser approval
Actor	Visitor / Exhibitor (register); Administrator / reviewer role (decide)
Preconditions	Account is `EmailVerified`; registration is open.
Main Flow	1. User picks registration type, Visitor / Other / Exhibitor. 2. Enter personal data (Arabic four-part name, English/passport name, nationality, DOB, place of birth). 3. Enter identity: Saudi national-ID (10 digits, starts `1`) or non-Saudi passport/Iqama (Iqama 10 digits, starts `2`). 4. Enter contact numbers (stored as entered, no format check). 5. Upload ID image (encrypted at rest, AES-GCM, magic-byte gate). 6. Identity photo verification (women-exception alternative path). 7. Exhibitor branch: organisation data plus companions. 8. Select venue track. 9. Consent to terms and submit to `RegistrationRequest`, account to `PendingApproval`, confirmation message, operation-log entry. 10. Reviewer opens the requests queue, reads data plus attachments. 11. Approve: set final user type, account to `Approved`, mint 12-char Crockford QR id at this transition, issue `Badge` (`SIMF-2026-xxxx` plus colour), notify.

Alternative Flows	A1 Exhibitor approval (UC-21): PR approves and assigns booth in one step. A2 Bulk approve via select-all. A3 On-site registration/reprint at the desk (UC-34). A4 User proposes a `ProfileType` at sign-up; admin assignment wins.
Exception Flows	Missing mandatory field / no terms consent to `VALIDATION_FAILED` (submission blocked). Reject to mandatory 10-500 char reason, account to `Rejected`. Re-approve/re-reject a non-pending user to 409 `ADMIN_USER_NOT_PENDING`. Unknown nationality code to `VISITOR_NATIONALITY_UNKNOWN`. Registration closed to submission refused with a clear message.
Postconditions	Account `Approved` with QR id + badge, or `Rejected` with recorded reason; audit rows (`Admin.VisitorApproved`/`VisitorRejected` etc.) carry actor + subject.

Companion: UC-07 status tracking, UC-31 open/close registration, internal-user onboarding plus TOTP enrolment, bulk admin actions.

Module 3, Badge & Access Control / Gate

UC-33 / UC-35, Gate scan (venue entry and hall arrival)

Field	Value
Use Case ID	UC-33 (venue entry), UC-35 (hall check-in)
Name	Scan a badge at a gate
Actor	Gate operator / Staff (or a fixed door device)
Preconditions	Operator authenticated with `Gates.Operate` and assigned to the gate; gate `IsActive`.
Main Flow (13-step engine)	1. Caller authenticated. 2. Caller has `Gates.Operate` + is an active assignee. 3. `IQrResolver` resolves the QR to a `UserProfile`. 4. Idempotency-key check (replay returns prior outcome). 5. Gate is active. 6-8. Holder is `Approved`, not Disabled, not Locked. 9. Holder's `ProfileType` is active. (9.5 reserved time-window no-op.) 10. Resolve direction (In/Out/Both, infer from last allowed scan at this gate). 11. Allow-list check on `GateProfileTypeAllow` (empty = general pass; filtered-empty = deny-all safe default). (11.5 reserved booking-required no-op.) 12. 5-second duplicate-window absorption keyed (GateId, UserProfileId). 13. Persist `GateScan` (`Outcome = Allowed`, resolved direction, server clock); a denial persists `Outcome = Denied` + `DenialReasonCode` and emits one `OperationLog` `GateScanDenied` row.
Alternative Flows	A1 Hall arrival (UC-35): QR at the door records `HallAttendance` enter (`Method = QrScan`); a GPS geofence crossing records `Method = Geofence`; both means update one row. A2 Attendee-to-attendee scan (UC-12): verified QR to `SavedContact`. A3 "Currently inside" derived from the most-recent allowed scan across all gates.

Exception Flows	`QR_UNKNOWN`, `GATE_INACTIVE_AT_SCAN`, `HOLDER_NOT_APPROVED`/`HOLDER_DISABLED`/`HOLDER_LOCKED`, `PROFILE_TYPE_INACTIVE`, `PROFILE_TYPE_NOT_ALLOWED` (all recorded). `IDEMPOTENCY_KEY_CONFLICT` (409). Failure-rate circuit: 10 or more denials / 60 s, 429 `GATE_FAILURE_CIRCUIT_OPEN` for 5 min.
Postconditions	Append-only `GateScan` row (INSTEAD-OF trigger refuses mutation; excluded from `RowAudit`); attendance/statistics and Q&A arrival-gating fed (section 5.5).

Companion: UC-11 view badge + QR (section 5.1). Note: GPS geofence hall-arrival chain has open items (retention, geofence on `Hall`) and is a deferred programme item; the design is as above.

Module 4: Bookings & Attendance

UC-09: Reserve a session seat (auto-confirm; gate check-in confirms). UC-22 approve/reject retained but dormant.

Field	Value
Use Case ID	UC-09 (book), UC-10 (cancel), UC-22 (approve)
Name	Reserve a seat (auto-confirm; provisional hold until gate check-in)
Actor	Visitor (approved). (Legacy PR/reviewer approval retained but dormant.)
Preconditions	Attendee is `Approved`; session open for booking with an available seat.
Main Flow	1. From session detail the attendee opens the hall seat map (available/held/taken). 2. Attendee selects an available seat. 3. System checks the no-overlap rule (no Pending/Approved booking for a time-overlapping session) and that the seat is still free. 4. Create `SeatReservation` in `Approved` (reservation-only auto-confirm), hold the seat (filtered unique index: one active reservation per seat per session). 5. Show the attendee an inline success message. 6. At the hall gate the attendee checks in (staff QR scan), marking the held seat confirmed; a pre-start sweep releases any hold not checked in. (No reviewer queue / BookingConfirmed notification on this path.)
Alternative Flows	Note the legacy Reject / Bulk approve actions are retained but dormant (no attendee booking enters Pending to act on); attendee-facing release happens via the pre-start uncheck-in sweep and cancellation. A3 Cancel (UC-10) a Pending/Approved booking before start, to `Cancelled`, seat released. A4 Random / open-seating reservation kinds (`SeatReservationKind`).
Exception Flows	Overlap: blocked with explanation. Seat taken

	meanwhile: constraint violation handled gracefully ("choose another"). Session full: no seats offered. Cancel after start: refused. Reject without reason: blocked.
Postconditions	Booking in a terminal/held state; seat map consistent; attendance later read from `HallAttendance`; "session-started-no-attendance" events raised for Notifications.

Module 5: Engagement / Q&A

UC-14 / UC-36: Ask a question and moderate it

Field	Value
Use Case ID	UC-14 (ask), UC-36 (moderate); UC-13 watch, UC-15/UC-26 comment
Name	Session question, 3-stage pipeline
Actor	Visitor (ask); Scientific committee ("team"); Moderator (per-session desk)
Preconditions	Session exists; attendee is Approved and has a `HallAttendance` enter record for the session; question window open (5 min before `StartUtc` to `EndUtc`).
Main Flow	1. Attendee opens the ask card and picks a recipient (speaker or host). 2. Attendee submits the text; backend sets `Phase` (Pre/Live) from the session start; question recorded `Pending`. 3. Stage 1: AI advisory filter sets `AiFilterVerdict` (config-gated; default stub; never auto-hides). 4. Stage 2: Scientific-committee "team" reviews in CP `/admin/question-queue`: approve / hide / escalate-to-role. 5. Stage 3: assigned Moderator runs the app desk for their session only: view, order, hide, put on-air, mark answered; stream updates live.
Alternative Flows	A1 Pre-session mode (B): the same ask card shows a distinct "ask before the session" label while upcoming; flows through the identical pipeline with `Phase = Pre`. A2 Comments (UC-15/UC-26): every comment gets an AI verdict (Passed/Flagged, never discarded) then admin approve/discard; approved appears in the feed. A3 Watch live (UC-13): stream + caption language + an optional notification to users near the venue. A4 Ratings triggers on session-end, hall-departure, live-close, day/programme-end.
Exception Flows	No hall-arrival record: composer not offered (gate, by design). Outside the window: questions closed. Moderator opens an

	unassigned session: no access. AI provider unavailable: advisory `ai-unavailable`, pipeline unaffected.
Postconditions	`SessionQuestion` progresses `Pending` to `Approved` or `Hidden` (an approved question is pushed on stage via `IsPushed`); moderator/admin actions written to the operation log; comment feed reflects only admin-approved comments.

Remaining modules: per-module use-case index

Full flows for these modules live in their governing functional design specifications; the design here does not restate flows beyond what each specification defines. Note: audience-comments were removed from the app, though the comment feature and Control Panel moderation remain.

Module	Governing FDS	Key use cases (actor)
Programme	FDS 004 Forum Programme	UC-23 Manage sessions; UC-24 Manage halls/seating; UC-25 Manage speakers + presentations (Admin/Scientific); UC-08 Browse agenda + search + add-to-calendar + reminders (Public/Visitor).
Exhibition	FDS 006 Exhibition	UC-18 Browse booths + venue map + navigate (Public/Visitor); manage booth record post-approval (Admin); exhibitor lead scan (Staff).
Sponsors	FDS 006 Exhibition	UC-18 View sponsors by tier (Public/Visitor); manage sponsors (Admin).
News / Media / Archive	FDS 010 Media, News & Archive	UC-32 Manage + read Media Center, News + categories, gallery, media partners, previous editions + visibility (Admin manage; Public read).
Networking & Cognitive AI	FDS 008 Networking & Cognitive AI	UC-16 Request one-to-one meeting (Visitor to PR approval); UC-17 AI assistant over two-level FAQ; meet-people-like-you matchmaking (>=80% recommend/push); AI session summary; accessibility AI (sign-language/speech/captions).
Meetings, B2B/B2C	FDS 013 Business Meetings	Configure hall/tables + reserve over a slot + schedule 2+ parties then Confirmed

		(Admin, built); attendee meeting-in-hall request then admin review vs hall availability then speaker double-opt-in (Visitor, build-ready).
Notifications	FDS 009 Notifications	UC-19 Receive a notification across in-app/email/SMS/WhatsApp; in-app inbox; reminders; channel-mix as configuration (Any signed-in; providers deferred).
Statistics & Dashboards	FDS 011 Statistics & Dashboards	UC-30 View per-day / overall / live-attendance / GPS-presence dashboards (Admin; figure list open).
Configuration	FDS 012 Control Panel Configuration	UC-28 Manage content blocks, categories, labels, colours, venue tracks (Admin); UC-31 open/close registration; view the operation log.
Contacts & Sharing	FDS 014 Contacts & Sharing	Share my contact via QR / vCard (consent-by-action); scan/resolve then save to My Contacts (Visitor); shared `Contact` directory referenced by org entities (Admin-curated).

The exhaustive per-page and per-endpoint realisation of every use case above (routes, permissions, buttons, bilingual states) is maintained in the per-page reference documentation and the Gherkin end-to-end test catalogue.

4. User Interfaces

SIMF presents three distinct clients over the one backend API: the Control Panel (internal administration, Blazor Server), the public Website (Blazor SSR plus interactive auth islands), and the Mobile application (Flutter, Android and iOS). This section catalogues the screens of each client, it does not reproduce every wireframe. The authoritative per-page detail (fields, buttons, validation, bilingual toast text, permissions, API calls) lives in the per-page reference documentation and its end-to-end catalogue files, indexed by route; annotated visual wireframes live in Figma (node ids are recorded per screen). The tables below reproduce the Real rows of the page index. No routes are invented here; every route is copied from that index.

4.1 Control Panel screens

The Control Panel is a Blazor Server application (interactive-server render, prerendering disabled) served at `/admin/*` under cookie authentication. Every list page is built on the shared SimfDataGrid (filter, select-all, row checkbox, quiet per-row icon actions) with the common CRUD framework

(`CrudShell`, `CrudPageFrame`, `CrudDialogFrame`, `CrudFormBase`), dialog-vs-full-page being a per-page user preference (CpPreferences in localStorage). Navigation is organised into ~12 permission-gated groups (Overview, People, Access Control, Programme, Scientific Committee, Exhibition, Engagement, Knowledge, Content, Public Relations, Gates, Reference Data). Localisation is via `IStringLocalizer<Strings>` with `dir` set from the current UI culture (bilingual Arabic/English, RTL); colours and typography come from `theme.tokens.css`. Every `/admin/*` page and action is gated by a `PermissionCatalog` code: the "Administrator" audience below denotes the baseline role that holds the page's permission; the exact code per page is in the per-page documentation and enforced by build-failing tests.

The following table lists the Control Panel pages. The single stub route `/m/{module}` (ModulePlaceholder) is omitted.

Route	Screen name	Audience	Purpose
Overview			
`/`	Dashboard	Any signed-in CP user	CP landing / overview dashboard
People & accounts			
`/admin/admins`	Admins	Administrator	Manage administrator accounts
`/admin/admins/pending`	Pending admins	Administrator	Approve/reject pending admin registrations
`/admin/others`	Other users	Administrator	Manage non-visitor/non-admin accounts
`/admin/others/pending`	Pending others	Administrator	Approve/reject pending "other" accounts
`/admin/visitors`	Visitors	Administrator	Manage visitor accounts
`/admin/visitors/pending`	Pending visitors	Administrator	Review + approve visitors (all-data view, tier-on-approve, photo viewer)
`/admin/visitors/vip`	VIP registration	Administrator	Register VVIP/VIP (Mawj fields, VIP photo); creates pending account
`/admin/visitors/vip/export`	VIP export	Administrator	VVIP/VIP welcome roster, JSON/CSV/Excel export
`/admin/delegates`	Delegates	Administrator	Delegate = visitor + IsDelegate + invited country; single add + bulk badge

			generation
`/admin/attendees`	Attendees	Administrator	Attendee directory
`/admin/print-bag`	Print bag	Administrator	Badge/print bag queue
`/admin/interests`	Interests	Administrator	Manage interest lookup
`/admin/profile-types/visitor`	Visitor profile types	Administrator	Manage visitor profile types
`/admin/profile-types/other`	Other profile types	Administrator	Manage non-visitor profile types
`/admin/organisations`	Organisations	Administrator	Organisation lookup
`/admin/regions`	Regions	Administrator	Region lookup
`/admin/contacts`	Contacts	Administrator	Shared contact records
`/admin/countries`	Countries	Administrator	Country lookup (incl. invited flag, delegation head/dates)
`/admin/vips`	VIPs	Administrator	VIP directory
`/admin/invitations`	Invitations	Administrator	Manage invitations
`/admin/reset-2fa`	Reset 2FA	Administrator	Reset a user's second factor
`/admin/roles`	Roles	Administrator	Manage roles
`/admin/roles/{id}/permissions`	Role permissions	Administrator	Assign permission codes to a role
Programme & sessions			
`/admin/themes`	Themes	Administrator	Programme themes
`/admin/halls`	Halls	Administrator	Halls (capacity, seat mode)
`/admin/halls/seat-layouts`	Seat layouts	Administrator	Per-hall seat layout editor
`/admin/speakers`	Speakers	Administrator	Manage speakers
`/admin/speaker-presentations`	Speaker presentations	Administrator	Speaker presentation files
`/admin/sessions`	Sessions	Administrator	Manage sessions
`/admin/sessions/seat-plans`	Session seat plans	Administrator	Per-session seat plan
`/admin/session-categories`	Session categories	Administrator	Session category lookup
`/admin/programme-days`	Programme days	Administrator	Programme day definitions
`/admin/session-moderators`	Session moderators	Administrator	Assign session moderators
`/admin/programme/timeline`	Programme timeline	Administrator	Timeline view of the programme
`/admin/bookings`	Bookings	Administrator	Seat-booking review queue (legacy approval workflow, retained but dormant; attendee

			bookings auto-confirm so the queue is empty).
`/admin/speaker-meeting-requests`	Speaker meeting requests	Administrator	Review speaker meeting requests
`/admin/speaker-availability`	Speaker availability	Administrator	Team-defined availability windows to VIP free slots
`/admin/hall-availability`	Hall availability	Administrator	Team-defined hall meeting-time windows to free slots
`/admin/delegation-meetings`	Delegation meetings	Administrator	Delegation-to-delegation meeting review desk
`/admin/document-requests`	Document requests	Administrator	Participation-document request review (Accept/Reject + note)
`/admin/badge-requests`	Badge requests	Administrator	Badge-update request review (Accept applies title to profile)
`/admin/meeting-tables`	Meeting tables	Administrator	Business-meeting tables
`/admin/business-meetings`	Business meetings	Administrator	Manage business meetings
Engagement, Q&A & attendance			
`/admin/question-queue`	Question queue	Administrator	Moderate the audience question queue
`/sessions/{id}/moderate`	Session moderation	Session moderator	Live moderate a session's questions
`/admin/ratings`	Ratings	Administrator	Rating responses + KPIs
`/admin/rating-config`	Rating config	Administrator	Dynamic rating-form configuration
`/admin/session-summaries`	Session summaries	Administrator	Author/publish session summaries
`/admin/hall-arrivals`	Hall arrivals	Administrator / operator	Hall arrival tracking
Exhibition			
`/admin/companies`	Companies	Administrator	Company records
`/admin/exhibitors`	Exhibitors	Administrator	Manage exhibitors
`/admin/booths`	Booths	Administrator	Manage booths (officer, company, map coords)
`/admin/sponsors`	Sponsors	Administrator	Manage sponsors

			(tier)
`/admin/media-partners`	Media partners	Administrator	Manage media partners
`/admin/venue-map`	Venue map	Administrator	2D venue-map nodes
Content & media			
`/admin/news`	News	Administrator	Manage news items
`/admin/media`	Media	Administrator	Media gallery items
`/admin/archive`	Archive	Administrator	Past-edition archive
`/admin/media-library`	Media library	Administrator	Unified image-asset library
`/admin/banners`	Banners	Administrator	Site banners
`/admin/content-blocks`	Content blocks	Administrator	CMS content blocks
Knowledge & AI			
`/admin/faq`	FAQ	Administrator	FAQ groups + entries
`/admin/ai`	AI dashboard	Administrator	AI overview dashboard
`/admin/ai/services`	AI services	Administrator	AI service list
`/admin/ai/services/{feature}`	AI service detail	Administrator	Per-feature AI service config
`/admin/ai/prompts`	AI prompts	Administrator	Prompt management
`/admin/ai/invocations`	AI invocations	Administrator	AI invocation telemetry
Access control & system			
`/admin/gates`	Gates	Administrator	Manage gates
`/admin/gates/operator`	Gate operator	Gate operator	Operator entry/exit scanning console
`/admin/gates/dashboard`	Gate dashboard	Administrator	Gate activity dashboard
`/admin/configuration`	Configuration	Administrator	System settings (key/value)
`/admin/email/templates`	Email templates	Administrator	DB-backed override editor for the 6 transactional identity emails
`/admin/organization-profile`	Organisation profile	Administrator	Edition/organisation profile + about content
`/admin/contact-inquiries`	Contact inquiries	Administrator	Public "contact us" inbox
`/admin/operations`	Operations	Administrator	Operational toggles (e.g. registration gate)
`/admin/operation-log`	Operation log	Administrator	Security-relevant business-event log
`/admin/logs`	Logs	Administrator	System logs
`/admin/statistics`	Statistics	Administrator	Event statistics
`/admin/attendance`	Attendance	Administrator	Attendance records

CP auth / account / framework pages (not in main nav): the sign-in flow (`/login`, `/login/totp`, `/login/recovery`, `/forgot-password`, `/auth/pending`, `/auth/rejected`), the account pages

(`/account/profile`, `/account/notifications`, `/account/totp-pairing`), deep-link create fallbacks (`/admin/{admins|others|visitors}/new`), and the framework error pages (`/Error`, `/not-found`, `/not-permitted`). These are reached via redirect or menu only; full detail is in the per-page CP documentation.

4.2 Website screens

The public Website is Blazor SSR for public content (rendered statically for speed) with interactive islands (`InteractiveServerNoPrerender`) for the auth and account flows; cookie auth (`simf.web.auth`) uses the same token-refresh handler as the CP, and public reads go through `SimfPublicClient`. Authorisation is binary (signed-in vs anonymous): the website has no per-page permission policies. There is no public navigation menu: every page is reached by direct URL or auth redirect. The public pages share the bilingual AR/EN RTL design system and the `ln-` SSR component kit noted per route in the page index.

Route	Screen name	Audience	Purpose
`/` (+ `/landing`)	Landing	Public	Marketing landing (bilingual Bootstrap SSR rebuild)
`/account`	Home (account)	Any signed-in	Signed-in home
`/programme`	Programme	Public	Public programme listing
`/speakers`	Speakers	Public	Public speaker listing (live data)
`/sessions/{id}`	Session detail	Public	Public session detail + downloads
`/visit`	Visit	Public	Visitor information page
`/login`	Login	Anyone	Sign-in (auth-only)
`/login/verify`	OTP verify	Mid-sign-in	E-mail OTP second factor (auth-only)
`/forgot-password`	Forgot password	Anyone	Request password reset (auth-only)
`/reset-password`	Reset password	After forgot	Set new password (auth-only)
`/account/profile`	Account profile	Any signed-in	View/edit profile (interactive)
`/account/notifications`	Notifications	Any signed-in	Notification list
`/account/pending`	Account pending	Pending account	Pending-approval state banner
`/account/rejected`	Account rejected	Rejected account	Rejected state banner
`/meeting/confirm`	Meeting confirm	Public (token)	Token-based meeting confirmation

The retired old static landing (`index.html`, removed 2026-07-14) is not listed. Per-page detail lives in the per-page website documentation.

4.3 Mobile application screens

The Flutter app (`src/Mobile/simf_app`; Riverpod + go_router + Dio) presents a persistent 5-tab bottom-nav shell (`StatefulShellRoute.indexedStack`), Home, Sessions, Badge, Map, Profile, with per-tab state preserved and auth/role gates (staff/moderator) redirecting appropriately. The theme is navy-always (dark pinned), bilingual with automatic RTL for Arabic, and every in-app page carries a global app-bar invariant (notifications bell + language toggle + dark-mode indicator + hamburger). There are ~34 feature folders under `lib/features/`; the page index enumerates each screen with its Figma node, backing `/app/*` endpoint and clean-code freeze decision. The table groups the built mobile screens by feature area (a superset of those folders); per-screen field detail and wireframes live in Figma and the per-page mobile documentation.

Screen (route)	Audience	Purpose
Onboarding / entry		
`splash` (#1)	Guest	Launch + server version-policy update gate
`onboarding` (#2)	Guest	3-step first-run carousel
`guestMode` (#12)	Guest	Guest-mode landing
Authentication & registration		
`signIn` (#3)	Guest	Sign-in
`verifyOtp` (#3a)	Mid-sign-in	Sign-in 2FA e-mail OTP
`forgotPassword` (#3b)	Guest	Request reset OTP
`resetPassword` (#3c)	After forgot	OTP + new password
`signUpForm` (#5)	Guest	Account sign-up form
`emailOtp` (#6)	Guest	Sign-up e-mail verification
`signUpVisitor` (#7)	Visitor	Profile data (type + lookups)
`signUpInterests` (#7-01)	Visitor	Interests + single profile save
`registrationSuccess` (#10)	Visitor (pending)	Terminal sign-up confirmation
`registrationStatus` (#11)	Visitor (pending)	Account approval status
`identityVerification`	Visitor (approved)	Avatar liveness capture
`biometricStepUp` (#7a)	Visitor+ (approved)	Enrol biometric via emailed OTP
`badgeSignIn`	Guest (badge holder)	Badge-QR sign-in entry
`badgePassword`	Guest (badge holder)	Badge-QR sign-in password step
`badgeActivation`	Guest (badge holder)	Passwordless badge activation
Home / profile / info		
`home` (#13)	Guest+	Home dashboard
`myArea` (#14)	Visitor	My-area dashboard (+ .ics/.vcf)
`more` (#41)	Guest+	Profile card + grouped nav + sign-out
`badge` (#32)	Signed-in	QR entry badge
`notifications` (#33)	Signed-in	Notification list
`accessibility` (#38)	Guest+	Accessibility settings (app-wide)
`aboutForum` (#37)	Guest+	About the forum (mission/vision/themes)
`aboutApp` (#207)	Guest+	About the app + manual

		update check + support
`forumGuide` (#200)	Guest+	Static forum guide
`faq` (#201)	Guest+	Public FAQ accordion
`contactUs` (#203)	Guest+	Contact form + org-profile panel
`terms` (#9)	Guest	Terms content
`chatbot` (#36)	Guest+	AI-assistant shell (interim, no API)
Programme & sessions		
`sessions` (#16)	Visitor+	Agenda (day strip + type tabs)
`sessionDetail` (#17)	Guest+	Session detail (+ seat card)
`mySeat` (#18)	Visitor	My reserved seat
`seatPicker`	Visitor	Seat selection / random reserve
`joinSessionHub`	Visitor	Join-session hub
`liveBroadcast` (#25)	Login-only	Live broadcast (YouTube + sign-language + captions)
`aiSummary` (#34)	Guest+	AI session summary
`sessionSummaryList` (#111)	Guest+	Searchable session-summary list
`sessionPresentations` (#202)	Approved	Day-tabbed session/presentation list
`myAreaSessions` (#113)	Approved	My-sessions (4 tabs)
`sendQuestion` (#26)	Visitor	Submit an audience question
`rate` (#40)	Visitor	Dynamic feedback/rating form
Speakers		
`speakers` (#19)	Guest+	Speaker list
`speakerProfile` (#20)	Guest+	Speaker profile (+ meeting request)
Exhibition & delegations		
`venueMap` (#15)	Guest	2D venue map + booths
`booths` (#22)	Guest+	Booth list
`boothMap`	Guest+	Booth located on the venue map
`exhibitorDetail` (#220)	Guest+	Exhibitor detail
`sponsors` (#23)	Guest+	Sponsor list
`sponsorDetail` (#221)	Guest+	Sponsor detail
`delegations` (#21)	Guest+ (public)	Invited-country delegations
Media, news & archive		
`news` (#29)	Guest+	News hub (latest-updates tab)
`gallery` (#30)	Guest+	Media gallery (photos/videos)
`mediaPartners` (#31)	Guest+	Media-partner grid
`archive` (#24)	Guest+	Past-edition archive
`archiveDetail` (#24-01)	Public	Archive edition detail
Networking, contacts & meetings		
`meetPeople` (#35)	Visitor	Smart match suggestions
`shareMyContact`	Visitor (approved)	Share contact QR / vCard

`scanContact`	Visitor (approved)	Scan & save a contact
`myContacts`	Visitor (approved)	Saved contacts
`meetings` (#116)	VIP (approved)	VIP two-party meetings page
`requests` (طلباتي)	Visitor (approved)	Unified requests feed (document/badge/meeting)
Staff / moderator / exhibitor		
`sessionModerate`	Moderator+ (per-session grant)	Moderator Q&A console
`gateScanner`	Staff (GateOperator grant)	Staff gate scan console
`staffRegisterVisitor`	Staff (Visitors.RegisterOnsite)	Walk-in visitor registration
`myVisitors` (زواري)	Exhibitor (approved)	Captured visitors list
`scanVisitor`	Exhibitor (approved)	Exhibitor lead-capture scan

Not silently dropped, removed/reserved screens: `signUpType` (#4, removed), `audienceComments` (#28, removed), `savedSessions` (#8, removed), and `myMeetings` (#115, removed) are recorded in the index but are not part of the shipped app and are therefore excluded from the catalogue above. The Flutter screens are an interim UI over the live `/app/*` API; the definitive per-screen build state, Figma node and E2E file for every row is in the page index and the per-page mobile documentation.

5. Module Description

This section is the narrative functional specification for each SIMF module. The modules map one-to-one to the approved Feature Design Specifications; the authoritative, exhaustive rule set for each remains its own feature design specification. For each module the description gives its purpose, key functionalities, business rules, integration points (APIs/services it exposes or calls), and error handling, grounded in the matching specification and cross-checked against the as-built backend feature areas.

The backend organises these modules as feature folders under `src/Backend/SIMF.Api/Endpoints/` (app surface `/api/v1/app/*`, admin surface `/api/v1/admin/*`) with matching infrastructure services under `src/Backend/SIMF.Infrastructure/`. Each module is presented across the mobile app, the Control Panel, and/or the website as its specification requires. Standard cross-cutting behaviour, the `ApiResult<T>` envelope, bilingual error messages, FluentValidation `VALIDATION_FAILED` (400), permission gating, and operation-log auditing, applies to every module and is not restated per module except where a module adds specific rules.

5.1 Authentication & Login

Module Purpose. Get a user into SIMF and keep them in: account creation, e-mail verification, sign-in with a second factor, session keep-alive (token refresh) and end (sign-out), and forgotten-password reset. It carries an account to `EmailVerified` and, for an approved user, issues a session.

Key Functionalities. Sign-up (creates a `Registered` account, e-mails a six-digit code); e-mail verification and code resend; password sign-in; a second factor: TOTP for users with an authenticator key paired, e-mailed OTP otherwise, with ten single-use Crockford recovery codes as a TOTP fallback; token refresh with rotation; sign-out; forgot/reset password; self-service change-password; TOTP enrolment (two-slot pattern) and admin-driven 2FA reset.

Business Rules. Password policy: at least 8 chars, at least 1 letter and 1 digit, not equal to the e-mail. Sign-in branches on account state: `Registered` refused (`AUTH_EMAIL_NOT_VERIFIED`), `EmailVerified`/`Approved` allowed, `PendingApproval` allowed with limited access on Web/App but refused on CP, `Rejected`/`Disabled` refused. Invalid-credentials and forgot-password responses are generic and never reveal account existence. Access token lives 5 minutes; the refresh token defines an absolute 24-hour session that rotation does not slide. A sign-in audience gate (`cp`/`web`/`app`) enforces that only CP-role holders sign in to the Control Panel and only visitors to Web/App. Account lockout and per-code attempt caps bound brute force.

Integration Points. Exposes the auth endpoints under `/app/auth/*`: `sign-up`, `verify-email`, `resend-code`, `sign-in`, `verify-totp`, `verify-otp`, `verify-recovery-code`, `refresh`, `sign-out`, `forgot-password`, `reset-password`, plus `/account/change-password` and the admin `/admin/staff/reset-two-factor`. Internally uses the token service (`IJwtTokenService`/`JwtTokenService`), the Identity store, the OTP/TOTP verifiers, and the e-mail channel of the Notifications module for verification and reset codes. Consumed by the Registration & Approval module (begins at `EmailVerified`) and by every client for session tokens.

Error Handling. Bilingual `ApiError` codes: `AUTH_EMAIL_ALREADY_REGISTERED` (409), `AUTH_CODE_INVALID`/`AUTH_CODE_EXPIRED` (400), `AUTH_ACCOUNT_NOT_FOUND` (404), `AUTH_INVALID_CREDENTIALS`/`AUTH_EMAIL_NOT_VERIFIED`/`AUTH_ACCOUNT_NOT_APPROVED`/`AUTH_ACCOUNT_DISABLED`/`AUTH_ACCOUNT_LOCKED`, `AUTH_MFA_TOKEN_INVALID`/`_EXPIRED`, `AUTH_TOTP_INVALID`, `AUTH_REFRESH_TOKEN_INVALID`/`_EXPIRED`, `AUTH_RESET_CODE_INVALID`/`_EXPIRED`, `AUTH_WRONG_SURFACE_CP`/`_WEB`, `RATE_LIMIT_EXCEEDED` (429). A reused rotated refresh token is rejected and logged as a possible theft signal.

5.2 Registration & Approval

Module Purpose. Carry an `EmailVerified` account through registration, the organisers' review, assignment of a final user type, and issue of an entry badge, for Visitor, "Other", and Exhibitor registrations.

Key Functionalities. Registration-type choice; personal, identity, contact and attachment capture; identity photo verification (with the documented women's-alternative path); the exhibitor organisation branch; terms consent and submission; registration-status tracking; Security review of visitors/"Other" and PR approval of exhibitors with booth assignment; final-user-type assignment and badge issue; on-site registration and badge reprint; registration open/close; and internal-user creation and first-sign-in TOTP enrolment. As-built adds admin-create-user with a 7-day invite code, the staff-vs-visitor CP page split, the `PendingApproval` to `Approved`/`Rejected` (reason 10 to 500 chars) workflow, the 12-char Crockford `QrId` minted on approval, the AES-256-GCM encrypted-at-rest ID image, and bulk admin actions.

Business Rules. Submission requires all mandatory fields and terms consent; the form branches by registration type and adapts identity fields Saudi-vs-non-Saudi. Saudi national ID = 10 digits starting `1`; Iqama = 10 digits starting `2`; passport free-form; phone numbers stored as entered, no format check. Phones are permissive; the ID image is a required attachment. `QrId` is minted at the

`PendingApproval` to `Approved` transition, not at create. Rejection requires a recorded reason. Registration closes automatically at the end of the last forum day, and a closed state blocks submission. The final user type set at approval drives permissions and app access; the `ProfileType.MobileAppRole` resolves the `mobile_app_role` JWT claim.

Integration Points. Exposes `/app/account/user-profile`, `/app/account/visitor-profile` (plus `/countries`, `/id-image`), `/app/account/profile-types`, and admin `/admin/staff` and `/admin/visitors` families (list, create, `{id}/approve`, `{id}/reject`, `pending/list`, bulk-delete/duplicate/import/export). Begins where Authentication ends; issues the badge the Badge & Access Control module operates; raises registration-submitted/approved/rejected/badge-ready events to Notifications. Encrypted ID image stored via `EncryptedUserIdDocumentStorage`; avatars via `IAvatarStorage`.

Error Handling. `AUTH_ACCOUNT_NOT_FOUND`, `ADMIN_EMAIL_ALREADY_REGISTERED`, `ADMIN_USER_NOT_PENDING` (409 on a re-approve/re-reject), `VISITOR_NATIONALITY_UNKNOWN`, and `VALIDATION_FAILED` with one detail per field. Uploads are magic-byte sniffed and size-capped (5 MB ID image, 2 MB avatar) and rejected before persistence. Bulk operations skip invalid targets (self-delete, admin-vs-admin) per row rather than failing the batch.

5.3 Badge & Access Control / Gates

Module Purpose. Operate the badge issued at approval: show it to the attendee, verify it at the venue, exchange attendee contacts, and record hall arrival, producing the attendance data statistics and engagement gating rely on.

Key Functionalities. The badge card and its signed-token QR; venue-entry verification by Staff scan; attendee-to-attendee contact exchange by badge scan; hall-arrival capture by QR-at-door plus GPS geofence; and the Gate Module, a first-class configurable access point with a 13-step ordered constraint engine, allow-lists, idempotency, a duplicate-absorption window, a "currently inside" derivation, and a failure-rate circuit breaker.

Business Rules. The badge QR is a signed token verified server-side; a copied or invented QR fails. A scan is allowed only for an active badge whose holder is `Approved`. Hall arrival is recorded by two means; if both fire for one attendee/session they update one `HallAttendance` row, not two, and one open attendance row per session is enforced. The gate engine runs its steps in a load-bearing order, short-circuiting to a recorded denial with a named `DenialReasonCode`; an empty allow-list passes, a filtered-empty allow-list denies all (safe-default rule); a 5-second window absorbs double-fire; 10 or more denials in 60 s opens a 5-minute circuit. GPS presence is reported on a bounded 30 to 60 s interval in small batches, not a continuous stream. `GateScan` is append-only.

Integration Points. Exposes `POST /app/gates/{gateId}/scans`, `GET /app/gates/my-reports/today`, and admin `/admin/gates` management; consumes `IQrResolver`, the idempotency store (`ScanIdempotency`), and hall/profile-type data. Reads `User`/`UserProfile`/`ProfileType`/`Hall`/`Session`. Feeds `HallAttendance`/`VenueEntry`/`GateScan`

to Statistics and the question-gating in Engagement; records `SavedContact` for the Contacts module.

Error Handling. Recorded denials: `QR_UNKNOWN`, `GATE_INACTIVE_AT_SCAN`, `HOLDER_NOT_APPROVED`, `HOLDER_DISABLED`, `HOLDER_LOCKED`, `PROFILE_TYPE_INACTIVE`, `PROFILE_TYPE_NOT_ALLOWED` (reserved: `OUTSIDE_TIME_WINDOW`, `BOOKING_REQUIRED_MISSING`). Non-recorded transport failures: 401 (unauthenticated), `GATE_OPERATOR_NOT_ASSIGNED` (403), `IDEMPOTENCY_KEY_CONFLICT` (409), `GATE_FAILURE_CIRCUIT_OPEN` (429). Every denial also emits one `OperationLog` row; successful scans stay in `GateScan` only.

5.4 Forum Programme & Sessions

Module Purpose. Define the event content, themes/pillars and sub-topics, halls and seating, speakers and their presentations, and sessions tying a theme, hall and speakers to a time, and present the attendee agenda. It is the content the bookings, live, engagement, AI-summary and statistics modules build on.

Key Functionalities. Control-Panel management of themes/sub-topics, halls with a generated seat grid, speaker profiles with presentation files, and sessions (title/description bilingual, theme, hall, category, start/end, one or more speakers each with a role, live/non-live flag); and the attendee agenda: day selector, search, session detail, speaker profile, add-to-calendar and device reminders.

Business Rules. All programme content is bilingual (Ar/En) and required in both languages. Saving a hall capacity generates one seat per position; reducing capacity removes only unallocated seats and warns on assigned ones. Each hall carries a geofence used by Badge & Access Control. A session's times must fall within the forum dates, end after start, and not overlap another session in the same hall; at least one speaker with a role is required. A session marked live is later streamed by Engagement. Programme records are soft-deleted; a theme in use is not removed while referenced.

Integration Points. Exposes `GET /app/programme/days`, `GET /app/programme/sessions/{id}` and admin `POST/PUT/DELETE /admin/sessions` (plus themes, halls, speakers, presentations). Reads `Category` (session category) and stores speaker photos/presentations via `ISpeakerPresentationStorage`. Provides sessions/halls/seats consumed by Bookings, Exhibition venue map, Engagement and Statistics; raises session-reminder events to Notifications.

Error Handling. `VALIDATION_FAILED` with per-field detail for missing bilingual fields, an end before start, a session outside the forum dates, or a hall already booked for the time window (a hall clash). Programme create/edit/deactivate is written to the operation log.

5.5 Bookings & Attendance

Module Purpose. Let an approved attendee reserve a session seat (auto-confirmed, held until gate check-in), and report whether the attendee actually attended, sitting on the Forum Programme and the hall-arrival records.

Key Functionalities. Making a booking from the seat map; the no-time-overlap rule; (Legacy) Control-Panel booking approval/rejection queue, retained but dormant; attendee reservations auto-confirm.;

cancellation before the session starts; the seat map and "My Seat" view; and session attendance derived from `HallAttendance`. The as-built seat service supports user booking, random assignment, admin-reserved rows and open seating.

Business Rules. The attendee must be `Approved`; the session must be open with a free seat; the session must not overlap a `Pending` or `Approved` booking of the same attendee. A booking is created `Approved` (reservation-only auto-confirm) and the seat is held; gate check-in confirms it (checked-in); a pre-start sweep releases any hold not checked in; cancellation is allowed only before the session start time. A seat cannot be held or confirmed twice for one session, enforced by a filtered unique index and a graceful constraint-violation handler ("that seat was just taken"); a held Pending seat expires if approval is slow. `Booking.Status` includes `Rejected`. Live counts are computed by aggregating attendance on a short cycle, not by incrementing a hot counter row.

Integration Points. Note the admin `/admin/bookings/{id}/approve|reject` endpoints (and BookingConfirmed/BookingRejected kinds) are retained but dormant; attendee reservations auto-confirm and raise no booking notification. Keep the session-started-no-attendance/no-entry events.

Error Handling. Overlap: blocked with a clear explanation; seat taken meanwhile: asked to choose again; session full: told no seats remain; cancel after start: refused; rejection without a reason: `VALIDATION_FAILED`. The uniqueness constraint violation under concurrency is caught and surfaced gracefully rather than as a 500.

5.6 Exhibition

Module Purpose. Present the exhibition: the booth directory, sponsors and their tiers, and the interactive 2D/3D venue map, so an attendee finds an exhibitor, learns about a sponsor and navigates the venue.

Key Functionalities. Control-Panel management of booth records (hall, number, logo, bilingual descriptor, booth contact), sponsors (bilingual name/description, logo, tier) and venue-map nodes; and attendee-facing browsing: a searchable booth directory with a directions action and dialler/mail links, sponsors grouped by tier, and a pan/zoom map with booth preview popups and in-venue navigation to a booth or an assigned seat. The as-built venue map uses 2D `VenueMapNode`s with a `Kind` and X/Y position.

Business Rules. A booth record exists once an exhibitor is approved and a booth assigned (Registration and Approval owns that); this module manages and presents it. A booth number is unique within its hall. A sponsor tier is one of the configured tiers (the naming source lists Strategic/Premium/Gold; the as-built `SponsorTier` enum is Platinum/Gold/Silver/Bronze). All content is bilingual; booth/sponsor/map content is public to read and authorised to change.

Integration Points. Exposes `GET /app/booths`, `GET /app/sponsors` and admin `POST /admin/exhibitors/list`, `POST /admin/sponsors`. Reads `ExhibitorProfile`/`Hall`/`Seat` and logo assets; the map guides to an assigned seat from Bookings and links a hall to its sessions from Forum Programme. Attendee position is handled under the location-privacy rules.

Error Handling. Duplicate booth number in one hall is rejected as not unique; missing bilingual fields return `VALIDATION_FAILED`. Create/edit/deactivate of booths, sponsors and map nodes is written to the operation log.

5.7 Engagement, Live, Q&A, Comments, Ratings

Module Purpose. Let the audience take part in a session: the live broadcast with AI translation/captions and an optional notification to users near the venue, the questions an attendee puts to a moderator, comments with two-stage moderation, and the multi-trigger session, day, programme and app ratings.

Key Functionalities. Start/stop live broadcast (YouTube via `youtube_player_iframe`, HLS/MP4 fallback) with a caption-language picker and an optional notification to users near the venue; a 3-stage Q&A pipeline (advisory AI filter, then Scientific-Committee CP queue, then per-session moderator desk) across Pre and Live phases; comment posting with an AI filter then admin review; and rating prompts fired end-of-day, end-of-programme, end-of-session (clock), on session-view leave, on hall departure and on live-stream close.

Business Rules. A session's questions open for an attendee only after they have a `HallAttendance` arrival record for the session and close at session end; the built window opens 5 minutes before start and closes at end. A question moves `Pending` to `Approved` or `Hidden` (an approved question is pushed on stage via `IsPushed`), set by a moderator who acts only on assigned sessions. Every comment passes AI then admin; the AI is advisory and never auto-approves or auto-hides: a flagged comment is queued, not discarded; only an admin-approved comment appears in the feed. The AI question filter is config-gated (`SessionQuestions:AiFilterEnabled`, default stub) and advisory only. An optional feature can notify users near the venue about live sessions; viewing the live stream is not geographically restricted. A rating fires once per session per user, deduplicated across triggers.

Integration Points. Exposes the app session-question, moderator-desk, comment and `/rate` endpoints and admin `POST /admin/session-comments/{id}/approve` and the Scientific-Committee `/admin/questions/queue`. Reads `Session`/`User`/`HallAttendance`; routes AI through the shared `IAiService`/cognitive-AI abstraction. Real-time updates are delivered over REST, polled by the client on a bounded interval. Raises live-session-starting and rating-request notifications to Notifications.

Error Handling. Question composer is hidden without an arrival record and after session end (by design, not an error); moderator access to an unassigned session's queue is refused. If the AI filter is unavailable, comments still queue for admin review (graceful degradation; the AI never auto-approves) and the question filter falls back to `ai-unavailable`.

5.8 Networking and Cognitive AI

Module Purpose. Connect attendees with one another and provide the cognitive-AI capabilities: interests and matchmaking, one-to-one meeting requests, the FAQ-backed AI assistant, the AI session summary, and the accessibility AI, all through one AI-provider abstraction.

Key Functionalities. Interest selection (a dynamic `Category`); "meet people like you" matchmaking with a match score and reason from shared interests/sessions, raising a recommendation plus push

at greater than or equal to 80%; one-to-one meeting requests routed to PR for approval; a conversational AI assistant backed by a two-level FAQ (groups to entries); AI session summaries stored as `SessionSummary`; and accessibility aids, sign-language and speech conversion, live captions, plus a CP AI-settings page. The as-built summary uses the central `IAiService`; the networking `Connection` and speaker/delegation `MeetingRequest` entities are shipped.

Business Rules. A meeting request needs a topic and an existing, Approved target other than the requester; a decline records a reason; the request moves `Pending` to `Approved`/`Declined` and, on approval, appears in both attendees' areas. Matchmaking exposes only what the other attendee's profile already shows; a recommendation triggers at a score greater than or equal to 80%. The AI assistant narrows to a FAQ group then an entry; FAQ groups/entries are bilingual and CP-managed. Every AI capability runs through the cognitive-AI abstraction: the provider is configuration, not code; only the data a capability needs is passed to the provider. The AI never acts as a final authority on comments (that gate belongs to Engagement).

Integration Points. Exposes `POST /app/account/connections`, `POST /app/programme/speaker-meeting-request`, `GET /app/my-meetings`, the AI-assistant/FAQ and AI-summary endpoints, and admin `POST /admin/business-meetings/list`, `POST /admin/delegation-meetings/list` and the FAQ/AI-settings pages. Reads `User`/`Session`/`Category`; shares the AI abstraction with Engagement's comment filter and captions. Raises match-recommendation and meeting notifications to Notifications.

Error Handling. A meeting request to an inactive/unknown/self target, or without a topic, is rejected with a clear bilingual error; a decline requires a reason. AI capabilities degrade behind the abstraction: a deferred/unavailable provider is a configuration state, and a real AI key must be provisioned before the real filter/summary path replaces the stub.

5.9 Notifications

Module Purpose. The single notification abstraction the rest of the system reaches users through: one operation ("notify this recipient about this event type, with this content") over four channels (in-app, e-mail, SMS, WhatsApp), plus the in-app inbox and reminders.

Key Functionalities. A calling feature raises an event; the service builds a `Notification` and sends it on the channels configured for its type, recording a `NotificationDelivery` per channel; the in-app inbox with unread count, filters, mark-read and deep-link; session and attendance reminders; and per-type channel-mix configuration. As-built, sending is asynchronous, a background worker drains the e-mail queue via MailKit, and a session-reminder worker dedups on `Session.ReminderSentUtc`.

Business Rules. A calling feature never chooses channels or formats channel messages: that is the single abstraction. The channel mix per type is configuration, not code; at least one channel per type. A notification is sent only to its intended recipient and in the recipient's language; verification/reset codes go by e-mail only. Sending is asynchronous so a user request never blocks on a gateway round-trip; a failed channel send is recorded and retried with bounded backoff behind a per-adapter circuit breaker. `NotificationKind` is a by-name-persisted enum extended additively (e.g. `BookingConfirmed=40`, `SessionReminder=41`, `MeetingScheduled=43`).

Integration Points. Exposes the in-app inbox reads and admin compose/channel-mix pages; consumes events from Authentication, Registration, Bookings, Engagement and Networking; sends via `SmtpEmailSender`/`EmailBackgroundService` and the (deferred) SMS/WhatsApp adapters. In-app push is read by the client from the in-app inbox over REST. The e-mail channel also carries Authentication's verification and reset codes.

Error Handling. A failed channel send is captured as a `NotificationDelivery` with a failure status and retried; a slow or unavailable gateway is bounded by adapter timeouts and a circuit breaker so it does not exhaust resources. Validation ensures a type from the catalogue, an existing recipient, at least one configured channel, and title and body in the recipient's language.

5.10 Media, News & Archive

Module Purpose. The content the forum publishes about itself and its history: the Media Center (media items, gallery, media partners), News items with categories, and the previous-editions archive with its post-event visibility control.

Key Functionalities. Control-Panel management of media items (kind: post, photo, video, social post, bilingual title and body, asset, publish date), media partners (name, logo), news items (bilingual title and body, image, publish date, category), and previous editions (title, brief, sessions, place, time, image, video, past speakers, edition statistics) with an `IsVisible` flag; and read-only presentation on the website and app: the gallery, social content, media-partner directory, news stream, and one archive card per year.

Business Rules. All content is bilingual and required in both languages; a news item needs an active news category and an image; a media item's asset must match its kind. An edition year is unique. The current edition (SIMF 2026) is hidden from the archive until the event ends, controlled by its visibility flag; past editions are visible. The media and news field sets and news categories are proposed for client confirmation. Social content is reviewed before publishing. Content is public to read and authorised to change.

Integration Points. Exposes `GET /app/news`, `GET /app/media` and admin `POST /admin/news`, `POST /admin/media/list` (plus media partners, previous editions). Reads `Category` (news categories) and image, video and logo assets via the media and asset storage services; the as-built archive owns `ArchiveMediaItem`, `ArchiveSessionTitle` and `ArchivePastSpeaker` and an `ArchiveVisibility` singleton.

Error Handling. Missing bilingual fields, a missing category or image, or a duplicate edition year produce `VALIDATION_FAILED` per field. Create, edit, deactivate and visibility changes are written to the operation log; an uploaded content-block image is type- and size-restricted.

5.11 Statistics & Dashboards

Module Purpose. The figures organisers watch before and during the forum, and the live picture of who is in the venue: per-day statistics, overall statistics, live attendance and GPS-presence movement, on the Control-Panel dashboard.

Key Functionalities. The dashboard page; per-day figures (registrations, badges printed, VIP and media badges, check-ins, approximate entries); overall figures (themes and topics, speakers, participating countries, registrations and badges, student badges, check-ins per day and total attendance, broadcast hours, audience questions); live venue and per-hall attendance; and GPS-presence movement and dwell-time views.

Business Rules. The Statistics context owns no source-of-truth records: it reads from the other contexts through their application services, never by cross-context queries. Figures are derived, never hand-entered (the only noted exception is student badges printed outside the CP, to confirm). A heavy figure is recomputed into a `StatisticSnapshot` on a fixed cycle of a few minutes; live figures are computed close to real time by aggregating `VenueEntry` and `HallAttendance` on a short cycle, not from a hot counter row. The dashboard reads GpsPresence rollups, never raw rows live; GPS-presence is sensitive personal data collected only with permission, encrypted at rest, under a confirmed retention rule.

Integration Points. Exposes `GET /admin/statistics` and reads (via owning-context services) `RegistrationRequest`, `Badge`, `VenueEntry`, `HallAttendance`, `Session`, `SessionQuestion`, `Theme`, `Speaker` and `Category`; uses `StatisticSnapshot` and `GpsPresence`. Live panels refresh over REST on a bounded interval.

Error Handling. A non-Dashboard role opening the dashboard route is refused (permission-gated); a stale live panel shows its last-updated time; a heavy figure requested twice is served from the snapshot on the second read. Location-permission-denied means the attendee's device contributes no GPS-presence data.

5.12 Control Panel Configuration

Module Purpose. The "everything dynamic" feature: let organisers change content, categories, labels, colours, venue tracks and key settings without a release, and record every change; split across a Content & Categories page (content team) and a System Configuration page (technical team), plus the Operation Log.

Key Functionalities. Dynamic content blocks (headings, texts, the in-app welcome message, banners, images, section labels, page content, bilingual, read at runtime); dynamic categories, labels and colours with add, hide and delete, display order and per-category colour; venue-track management; registration open and close; the system-configuration settings; and the Operation Log viewer. As-built config surfaces are `SystemSetting` (key/value), `OrganizationProfile`, `ContentBlock`/`Banner`, and the `RegistrationGate` singleton.

Business Rules. A change made here takes effect across the website and app at runtime with no release. A content block holds a key and a value in both languages; a category entry holds a bilingual name, colour, display order and `Kind`; a category in use is hidden, not hard-deleted. Registration closes automatically at the end of the last forum day. This module manages content and per-category colours only: it does not change the brand palette or typeface fixed by the visual identity and the theme tokens. The operation log is append-only from the application: no user, including an

Administrator, edits or deletes an entry. It does not cover roles and permissions, which are handled by the roles and permissions module.

Integration Points. Exposes admin `GET /admin/system-settings`, `POST /admin/operation-logs/list` and the content, category, venue-track and registration-control pages, plus `GET /app/site-settings` for the clients to read content at runtime. Every other module writes its configuration and business events to the operation log, which this module owns and views. Content-block images stored as assets.

Error Handling. A non-permitted user opening a configuration page is refused; editing a log entry is not possible (append-only); a category in use is hidden rather than removed so existing references hold. An uploaded content-block image is type- and size-restricted.

5.13 Business Meetings

Module Purpose. Admin-arranged B2B and B2C business meetings and the flexible per-hall configuration and allocation they sit on: an operator configures a hall's purpose and layout, reserves space over a time-slot, and schedules a meeting between two or more named parties at a reserved table; confirmed meetings surface on each participant's mobile dashboard.

Key Functionalities. Per-hall purpose (Booth, Session, Meeting) and layout; allocation by whole, random-by-count or row-column over a from-to slot; meeting-table definition with capacity; scheduling a confirmed meeting (B2B or B2C, group of 3 or more allowed); conflict enforcement; cancellation with participant notification; and the CP hall-config and Business Meetings pages. The v1.1 draft adds an attendee-initiated speaker/VIP request path with a hall-availability gate and a speaker double-opt-in e-mail, partly built, the rest owner-resolved and build-ready.

Business Rules. A meeting is created `Confirmed` (no `Pending`), admin-arranged only; the 1.0 core excludes an attendee request queue. Participants must be at least 2, distinct and each active/approved; a group must fit the table capacity. One active reservation per unit per overlapping slot (filtered unique index plus race guard) and no participant double-booking across overlapping meetings. Random-by-count allocation stops at the count or hall capacity, whichever is smaller. Each participant is a `Company` (App FK) or a visitor (bare `Guid`, no cross-DB FK, resolved on read). New tables landed additively. The v1.1 speaker path adds an `AwaitingSpeaker` state and an owner-approved `AllowAnonymous` 72-hour single-use action-token endpoint for the speaker's e-mail approve/reject links.

Integration Points. Exposes admin `/admin/meeting-tables`, `/admin/business-meetings`, `/admin/speaker-availability`, `/admin/speaker-meeting-requests/{id}/respond`, and app `POST /app/speakers/{id}/meeting-requests`, `GET /app/speakers/{id}/available-slots`, `POST /app/delegation-meeting-requests`. Reuses the seat-reservation service mechanics from Bookings; resolves visitor names via the Identity round-trip; raises `MeetingScheduled`/`MeetingCancelled` (and planned `MeetingRequestConfirmed`) notifications; feeds the Page_014 dashboard meeting union (not yet wired). The v1.1 speaker e-mail uses the MailKit e-mail path.

Error Handling. A participant inactive/not found, a table taken for an overlapping slot, or a participant already in an overlapping meeting produces a clear bilingual error; a group over table

capacity is rejected; a non-permitted admin gets 403/nav hidden. Notification failure never rolls back the meeting (swallow-and-log). A used/expired v1.1 action token shows a neutral "no longer valid" landing, never leaking state.

5.14 Contacts & Sharing

Module Purpose. Two deliberately separate mechanisms: (Track 1) a shared, de-duplicated `Contact` directory that org-facing entities (`Company`, `Sponsor`, `MediaPartner`, `Speaker`, `Booth` officer) reference by FK; and (Track 2) consent-by-action visitor-to-visitor contact sharing by QR or OS-share-intent vCard, saved to *My Contacts*.

Key Functionalities. Track 1: create/edit/soft-delete a `Contact` (logo, bilingual name, phones, fixed social set, website, map lat/long, country FK) and link it from any of the five org entities via a "link existing or create new" picker; one row reused across roles (editing once updates everywhere). Track 2: mint/rotate a dedicated share token, show it as a QR, resolve it to a live-projected card, save a `SavedContact`, list/remove *My Contacts*, and export a vCard 3.0 via the OS share sheet.

Business Rules. A visitor card and an org contact must not be the same record: a visitor card is projected live from `UserProfile` (plus a permitted bare-`Guid` Identity round-trip for e-mail), never copied into the org directory (two-DB rule). The share token is dedicated and separate from the entry `QrId`, so a gate scan never harvests a card; it is rotatable. A `Contact` is independent of its referrers, deactivating one still referenced by an active entity is blocked (`OnDelete.Restrict` plus guard). Org read projections prefer the linked `Contact` and fall back to inline columns so the shipped wire contract is preserved (append-only). Saving is idempotent per (owner, subject); you cannot save yourself; no PII snapshot is stored, resolved on read; no notification is sent to the subject. Social links are a fixed set (Facebook/X/LinkedIn/Instagram); logos are relative paths, never absolute URLs.

Integration Points. Track 1 exposes admin `Contacts.View`/`Contacts.Edit`-gated CRUD plus picker, wired into the five org admin forms, and a shared read-only card component on the website/app. Track 2 exposes app `/app/*` endpoints: `GET share-token`, `POST share-token/rotate`, `POST contacts/resolve`, `POST contacts/save`, `GET contacts`, `DELETE contacts/{id}`, `GET contacts/{id}/vcard`, app-only, no CP surface, gated by `RequireApprovedAccount` plus app audience like `Connection`. Reads `Country`, `UserProfile`, `Organisation`; does not replace `Connection` or the planned attendee-discovery directory.

Error Handling. Soft-deleting a still-referenced `Contact` is rejected with a bilingual error; resolving an unknown/revoked share token returns 404; saving yourself is rejected; a deactivated subject makes the saved row show a limited/unavailable card. Field validation aligns FluentValidation = EF = UI lengths; lat/long must both be set or both null within valid ranges; social URLs are absolute-URL validated.

6. Data Model

This section describes the SIMF logical data model. It is grounded in the authoritative Data Model and Database Design (v1.2, Approved), its conventions, bounded-context model, core ERD, indexing,

and Amendments A/B, and in the as-built entity/table listing. Where the data model states that exact column types, lengths and indexes live in the EF Core configuration and generated migrations, this document does not reproduce them; it presents the logical model and the sourced key columns, and points to the migrations for the exhaustive physical schema.

6.1 Conceptual View

The model is a logical, technology-agnostic view organised by the twelve bounded contexts the data model defines. Each context owns its own tables; a context that needs another's data reads it through that context's application service, not through a cross-context query. Standard columns (`Id`, `IsActive`, audit columns) are not repeated per entity.

Conventions. Every table has a GUID `Id` primary key. Tables and columns are PascalCase; tables are singular. Soft delete uses an `IsActive` boolean (list queries filter on it). Entities that change over time carry `CreatedAt`, `CreatedBy`, `ModifiedAt`, `ModifiedBy`. Foreign keys are named `<Entity>Id`. Fixed software-owned sets are enums; client-maintained sets (registration sub-types, session categories, interests) are data in the `Category` table, not enums. Bilingual text uses paired columns (`TitleAr`/`TitleEn`) rather than a translations table. Timestamps are stored in UTC.

Identity & Access

- **Entities:** `User`, `Role`, `Permission`, `RolePermission`, `UserRole`, `RefreshToken`, `EmailVerificationCode`, `TotpSecret`.
- **Relationships:** `User` has many `UserRole`; `Role` has many `RolePermission` and `UserRole`; `Permission` is one action on one page (the fixed catalogue) and has many `RolePermission`; `RefreshToken`, `EmailVerificationCode`, `TotpSecret` belong to `User`.
- `AccountState` enum: Registered, EmailVerified, PendingApproval, Approved, Rejected, Disabled.
- As-built: `User`/`Role`/`UserRole` are realised on ASP.NET Core Identity (`SimfUser: IdentityUser<Guid>`, `SimfRole`, `AspNetUserRoles`); `TotpSecret` is the Identity authenticator-key token store (no separate table); `EmailVerificationCode` is realised as `AccountCode` with a `Purpose` field.

Registration & Approval

- **Entities:** `RegistrationRequest`, `AttendeeProfile`, `ExhibitorProfile`, `Companion`, `Attachment`.
- **Relationships:** `RegistrationRequest` **belongs to** `User`, **has one** final-type `Category`, **has many** `Attachment`; `AttendeeProfile`/`ExhibitorProfile` **belong to** `User`; `Companion` **belongs to** `ExhibitorProfile`; `Attachment` **belongs to** `RegistrationRequest` and references an `Asset`. The final user type is a `Category` of kind VisitorSubType/OtherType set on approval.

Badge & Access Control

- **Entities:** `Badge`, `VenueEntry`, `HallAttendance`, `SavedContact`, `Gate`, `GateProfileTypeAllow`, `GateAssignment`, `GateScan`, `ScanIdempotency`.
- **Relationships:** `Badge` belongs to `User`; `VenueEntry` belongs to `Badge`; `HallAttendance` belongs to `User` and `Session` (records enter plus leave time by QR scan or GPS geofence); `SavedContact` links an owner `User` to a scanned `User`; `Gate` has many `GateProfileTypeAllow`, `GateAssignment`, `GateScan`. `GateScan` is an append-only audit log

(bigint identity PK; INSTEAD-OF trigger refuses mutation). The gate module uses logical cross-context FKs to Users/UserProfiles/ProfileTypes, enforced at write time by the gate services.

Forum Programme

- **Entities:** `Theme`, `SubTopic`, `Hall`, `Seat`, `Session`, `Speaker`, `SessionSpeaker`, `SpeakerPresentation`, `Booking`.
- Relationships: `Theme` has many `SubTopic` and `Session`; `Hall` has many `Seat` and `Session`; `Session` belongs to `Theme`, `Hall`, a session-category `Category`, and has many `SessionSpeaker`; `SessionSpeaker` links `Session` and `Speaker` (`RoleInSession` Speaker/Host); `SpeakerPresentation` belongs to `Speaker` and `Session`; `Booking` belongs to `User`, `Session`, `Seat` (reservation auto-confirmed on create; confirmed at gate check-in).

Exhibition

- **Entities:** `Booth`, `Sponsor`, `VenueMapNode`.
- **Relationships:** `Booth` belongs to `Hall` and `ExhibitorProfile`, references an `Asset` (logo); `Sponsor` references an `Asset` (logo), carries a `Tier`; `VenueMapNode` marks a hall/zone/booth (`Kind`, `PositionX/Y`). Delegations are not a standalone entity, a delegate is a visitor with `UserProfile.IsDelegate = true` and an invited `Country` (`Country.IsInvited = true`), two additive booleans, no new entity.

Engagement & Live

- **Entities:** `LiveSessionState`, `SessionQuestion`, `Comment`.
- **Relationships:** `LiveSessionState` **belongs to** `Session`; `SessionQuestion` **belongs to** `Session`, asked-by a `User`; `Comment` **belongs to** `Session` and an author `User` and always carries both an `AiResult` and an `AdminDecision` (two-stage moderation). A session's questions open only after the asking user has a `HallAttendance` enter record for that session.

Networking

- **Entities:** `Interest` (backed by a `Category` of kind Interest), `UserInterest`, `MeetingRequest`, `MatchSuggestion`.
- **Relationships:** `UserInterest` **links** `User` and an interest `Category`; `MeetingRequest` **links** a requester `User` and a target `User`; `MatchSuggestion` **links** a `User` to a suggested `User` (a score of 80 or more triggers a recommendation + push).

Content & Media

- **Entities:** `MediaItem`, `MediaPartner`, `NewsItem`, `Edition`, `EditionStat`, `EditionSpeaker`.
- **Relationships:** `Edition` has many `EditionStat` and `EditionSpeaker`, references cover/media `Asset`s; `NewsItem` belongs to a news-category `Category`; `MediaItem`/`MediaPartner` reference `Asset`s. Exact `MediaItem`/`NewsItem` field sets are confirmed per-feature.

Notifications

- **Entities:** `Notification`, `NotificationDelivery`.
- **Relationships:** `Notification` belongs to a recipient `User`; `NotificationDelivery` belongs to `Notification`, one delivery per channel (InApp / Email / SMS / WhatsApp).

Cognitive AI

- **Entities:** `FaqGroup`, `FaqEntry`, `AiSetting`, `SessionSummary`.
- **Relationships:** `FaqGroup` **has many** `FaqEntry` (the two knowledge levels); `SessionSummary` **belongs to** `Session`; `AiSetting` is a key/value row.

Analytics & Statistics

- **Entities:** `GpsPresence`, `StatisticSnapshot`.
- **Relationships:** `GpsPresence` belongs to `User`, batched append-only telemetry with a rolling retention purge. `StatisticSnapshot` stores computed dashboard figures; the context mostly reads from the others.

Platform Configuration

- **Entities:** `Category`, `ContentBlock`, `VenueTrack`, `RegistrationControl`, `Asset`, `OperationLog`.
- **Relationships:** the single `Category` table carries every dynamic client-maintained list (tagged by `Kind`, with its own colour); `Asset` is referenced wherever a file is stored; `OperationLog` **belongs to** the acting `User`. `RegistrationControl` is a single configuration row.

Note. The entity names above are the logical model. The as-built table names differ in places (e.g. `AttendeeProfile` to `UserProfile`, `VenueEntry` to gate/`HallAttendance` model, `Booking` to `SeatReservation`, `Comment` to `SessionComment`, `MeetingRequest` to `SpeakerMeetingRequest`/`BusinessMeeting`/`DelegationMeetingRequest`). The mapping is given in section 6.3 and the migrations are canonical.

6.2 ER Diagram

Two-database split. SIMF uses two physically separated SQL Server 2022 databases: `SIMF_Identity` (the Identity & Access tables, via `SimfIdentityDbContext`, migration history `__EFMigrationsHistory_Identity`) and `SIMF_App` (all other tables, via `SimfAppDbContext`, migration history `__EFMigrationsHistory_App`). Consequences by design:

- **No cross-database foreign keys.** Any App to Identity reference is a logical FK, a bare `Guid` enforced in application code, never a DB constraint (e.g. `UserProfile.UserId`, `GateScan.ScannedByUserId`).
- **No cross-database transaction**, a unit of work touches one context/database at a time.
- **No duplicated data**, Identity-owned data is resolved on read; the sole exception is the deliberate immutable audit-snapshot pattern (action logs capture the actor's display name/email at write time).

The split is a security boundary (Identity can be backed up / encrypted / access-controlled independently); the two connection strings may point at the same instance or **separate physical servers**.

Core ERD. The core diagram centres on `User` (Identity) and the programme/engagement entities in `SIMF_App`. Lookup, configuration and audit tables are omitted from the core diagram to keep it readable. Central keys: every entity has a GUID `Id`; `User.Email` and `Badge.ReferenceNumber` are

unique; `Permission` is `(Page, Action, Code)`; join tables (`UserRole`, `RolePermission`, `SessionSpeaker`, `UserInterest`, `NotificationDelivery`) carry composite links.

Core relationships (from the mermaid ERD):

Entity	Relates to	Cardinality	Via
User	UserRole	1 to many	user has roles
Role	UserRole	1 to many	role membership
Role	RolePermission	1 to many	role grants perms
Permission	RolePermission	1 to many	perm in roles
User	AttendeeProfile	1 to 0..1	user profile
User	ExhibitorProfile	1 to 0..1	exhibitor profile
User	RegistrationRequest	1 to many	submits
RegistrationRequest	Attachment	1 to many	includes
User	Badge	1 to 0..1	holds
Badge	VenueEntry	1 to many	records scans
User	HallAttendance	1 to many	records
User	Booking	1 to many	makes
Theme	SubTopic / Session	1 to many	contains / groups
Hall	Seat / Session	1 to many	contains / hosts
Session	SessionSpeaker	1 to many	features
Speaker	SessionSpeaker	1 to many	appears in
Session	Booking	1 to many	booked
Seat	Booking	1 to many	reserved
Session	LiveSessionState	1 to 0..1	has
Session	SessionQuestion / Comment	1 to many	receives
Session	SessionSummary	1 to 0..1	summarised
ExhibitorProfile	Booth	1 to many	runs
Hall	Booth	1 to many	holds
User	MeetingRequest	1 to many	requests
User	UserInterest	1 to many	picks
Edition	EditionStat / EditionSpeaker	1 to many	reports / lists
User	Notification	1 to many	receives
Notification	NotificationDelivery	1 to many	sent on
FaqGroup	FaqEntry	1 to many	contains
Category	UserInterest	1 to many	typed

The full core ERD is maintained in the data model (mermaid `erDiagram` source); the complete relationship set including lookup, config, and audit tables is documented there as well.

6.3 Schema Design

6.3.1 Data dictionary, key tables

The tables below are the highest-value tables. Type conventions: `GUID` = the standard `Id`/FK primary/foreign key type; `enum(int)` = integer-backed, C#-owned; `datetime(UTC)` = UTC timestamp; `string` = text whose exact length is fixed in the EF configuration/migration (lengths not

reproduced here to avoid fabrication). Standard audit columns (`CreatedAt/By`, `UpdatedAt/By`, `IsActive`, `DeletedAt`) apply per `BaseAuditEntity` and are not repeated except where load-bearing.

AspNetUsers (`SimfUser: IdentityUser<Guid>`), SIMF_Identity

Column	Type	Null	Key/Constraint	Description
Id	GUID	No	PK	User identifier
Email	string	No	Unique (User.Email)	Login / contact email (Identity)
PasswordHash	string	Yes	-	Provided by ASP.NET Core Identity
SecurityStamp	string	Yes	-	Identity stamp; drives immediate revoke
DisplayName	string	No	-	Shown name
AccountState	enum(int)	No	-	Registered to Disabled (no separate IsActive on User)
UserType	enum(int)	No	-	Visitor=0, (1 reserved), Admin=2
PasswordChangeRequired	bool	No	-	Force change on next sign-in
PasswordChangedAtUtc	datetime(UTC)	Yes	-	Password max-age tracking
LastSuccessfulSignInAtUtc	datetime(UTC)	Yes	-	Dormant-account sweep input
AvatarRelativePath	string	Yes	-	Relative path to avatar file

Note: lockout and two-factor state are the standard ASP.NET Core Identity columns on `AspNetUsers`.

Permissions (`Permission`), SIMF_Identity

Column	Type	Null	Key/Constraint	Description
Id	GUID	No	PK	Permission id
Code	string	No	Unique code	Permission code (`Page.Action`)
Page	string	No	-	Page the action belongs to
Action	string	No	-	Action on the page
DisplayName	string	No	-	Human-readable label

RolePermissions (`RolePermission`), SIMF_Identity

Column	Type	Null	Key/Constraint	Description
RoleId	GUID	No	PK part; FK to AspNetRoles	Role side of the link
PermissionId	GUID	No	PK part; FK to Permissions	Permission side of the link

Composite primary key `(RoleId, PermissionId)`; pure join table.

RefreshTokens (`RefreshToken`), SIMF_Identity

Column	Type	Null	Key/Constraint	Description
Id	GUID	No	PK	Token record id
UserId	GUID	No	FK to AspNetUsers	Owning user
TokenHash	string	No	-	Hashed token (never stored raw)
ExpiresAt	datetime(UTC)	No	-	Expiry
RevokedAt	datetime(UTC)	Yes	-	Revocation time
RotatedFromId	GUID	Yes	Self-ref	Rotation chain; reuse detection

AccountCodes (`AccountCode`), SIMF_Identity

Column	Type	Null	Key/Constraint	Description
Id	GUID	No	PK	Code record id
UserId	GUID	No	FK to AspNetUsers	Owning user
Purpose	enum(int)	No	AccountCodePurpose	EmailVerification/PasswordReset/SignInOtp/BadgeActivationOtp/BiometricEnrolStepUp
Code	string	No	-	The one-time code
ExpiresAt	datetime(UTC)	No	-	Expiry
ConsumedAt	datetime(UTC)	Yes	-	When redeemed
AttemptCount	int	No	-	Per-code attempt cap

UserProfile, SIMF_App. Bilingual `AttendeeProfile` as built.

Column	Type	Null	Key/Constraint	Description
Id	GUID	No	PK	Profile id
UserId	GUID	No	Unique; logical FK to AspNetUsers	Owning user (no DB FK, cross-DB)
(bilingual name)	string	No	-	Arabic + English name (paired columns)
Gender	enum(int)	Yes	-	Gender

NationalId / IqamaNumber / PassportNumber	string	Yes	Encrypted at rest	Identity numbers (PII, IPiiEncryptor)
(mobiles)	string	Yes	Encrypted at rest	Mobile numbers (PII)
OrganisationId	GUID	Yes	logical FK to Organisation	Organisation
ProfileTypeId	GUID	Yes	logical FK to UserProfileType	Profile type
IsDelegate	bool	No	-	Delegation flag
QrId	string	No	Unique	Badge QR identifier
ReferenceNumber	string	No	-	SIMF reference number
(VIP columns)	-	-	-	VIP profile fields

Session, SIMF_App

Column	Type	Null	Key/Constraint	Description
Id	GUID	No	PK	Session id
Code	string	No	-	Session code
HallId	GUID	No	FK to Hall	Hosting hall
CategoryId	GUID	Yes	FK to SessionCategory	Session category (dynamic)
Type	enum(int)	Yes	SessionType	Session type
StartUtc / EndUtc	datetime(UTC)	No	-	Start / end
Status	enum(int)	No	SessionStatus	Scheduled/Held/Recorded/Published
LiveStreamUrl	string	Yes	-	Live stream URL (YouTube)
LiveSignLanguageUrl	string	Yes	-	Sign-language feed URL
LiveCaptions (Arabic)	string	Yes	-	Live captions
(recording columns)	-	-	-	Recording metadata
ReminderSentUtc	datetime(UTC)	Yes	-	Session-reminder dedup guard

SeatReservation, SIMF_App (`Booking`)

Column	Type	Null	Key/Constraint	Description
Id	GUID	No	PK	Reservation id
(SessionId)	GUID	No	FK to Session	Session booked
(SeatId)	GUID	Yes	FK to Seat	Seat (null for open seating)
Kind	enum(int)	No	SeatReservationKind	UserBooking/RandomAssignment/AdminReservedRow/OpenSeating

Status	enum(int)	No	BookingStatus	Pending/Approved/Rejected
(review columns)	-	-	-	ReviewedByUserId / ReviewedAt / reason
-	-	-	Filtered unique index	One active reservation per seat per session

GateScan, SIMF_App (append-only audit log)

Column	Type	Null	Key/Constraint	Description
Id	bigint IDENTITY	No	Clustered PK	Monotonic, no fragmentation
GateId	GUID	No	logical ref to Gate	Gate scanned at
UserProfileId	GUID	Yes	logical FK to UserProfile	Scanned visitor
QrIdAtScan	string(12)	No	-	QR id preserved exactly (even after rotation)
Direction	enum(int)	No	CheckIn/CheckOut	Scan direction
Outcome	enum(int)	No	Allowed/Denied	Scan outcome
DenialReasonCode	string	Yes	-	Reason when denied
ScannedAtUtc	datetime(UTC)	No	Indexed	Server clock (authoritative)
ClientScannedAtUtc	datetime(UTC)	Yes	-	Device clock (client-asserted)
ScannedByUserId	GUID	No	logical FK to SimfUser	Operator
Source	enum(int)	No	Simulator/MobileApp/Kiosk	Scan source
CorrelationId	string	No	-	Request correlation
IpAddress / UserAgent	string	Yes	-	Client metadata
IdempotencyKey	string	Yes	Unique filtered (IdempotencyKey,GateId)	Replay enforcement

Mutation is refused by an INSTEAD-OF UPDATE/DELETE trigger; the table opts out of `RowAudit`. Its five to six non-clustered indexes are specified in the data model.

Notifications (`Notification`), SIMF_Identity

Column	Type	Null	Key/Constraint	Description
Id	GUID	No	PK	Notification id
UserId	GUID	No	FK to AspNetUsers	Recipient
Kind	enum(int)	No	NotificationKind (by name)	e.g. BookingConfirmed/SessionReminder/BookingRejected
Title /	string	No	-	Bilingual title

TitleArabic				
Body / BodyArabic	string	Yes	-	Bilingual body
Severity	enum(int)	No	NotificationSeverity	Info/Success/Warning/Error
ReadAt	datetime(UTC)	Yes	-	When read
RelatedEntityType / RelatedEntityId	string / GUID	Yes	-	Deep-link target

Note: the logical model separates `Notification` from `NotificationDelivery` (per-channel delivery). The as-built table places `Notification` in `SIMF_Identity`; the per-channel delivery model is retained in the logical data model.

6.3.2 Complete entity index

Every entity from the logical data model, by context. This is the exhaustive logical entity list; no entity is dropped. As-built table names (where they differ) are listed in the physical schema for each context.

Context	Entities
Identity & Access (section 5.1)	User, Role, Permission, RolePermission, UserRole, RefreshToken, EmailVerificationCode, TotpSecret
Registration & Approval (section 5.2)	RegistrationRequest, AttendeeProfile, ExhibitorProfile, Companion, Attachment
Badge & Access Control (section 5.3)	Badge, VenueEntry, HallAttendance, SavedContact, Gate, GateProfileTypeAllow, GateAssignment, GateScan, ScanIdempotency
Forum Programme (section 5.4)	Theme, SubTopic, Hall, Seat, Session, Speaker, SessionSpeaker, SpeakerPresentation, Booking
Exhibition (section 5.5)	Booth, Sponsor, VenueMapNode
Engagement & Live (section 5.6)	LiveSessionState, SessionQuestion, Comment
Networking (section 5.7)	Interest, UserInterest, MeetingRequest, MatchSuggestion
Content & Media (section 5.8)	MediaItem, MediaPartner, NewsItem, Edition, EditionStat, EditionSpeaker
Notifications (section 5.9)	Notification, NotificationDelivery
Cognitive AI (section 5.10)	FaqGroup, FaqEntry, AiSetting, SessionSummary
Analytics & Statistics (section 5.11)	GpsPresence, StatisticSnapshot
Platform Configuration (section 5.12)	Category, ContentBlock, VenueTrack, RegistrationControl, Asset, OperationLog

As-built additions. The delivered `SIMF_App` schema includes further additive entities, including: `ProgrammeDay`, `SessionTheme`, `SessionCategory`, `SessionModerator`, `SessionComment`, `SessionCommentLike`, `HallSeatLayout`, `Exhibitor`, `ExhibitorMembership`, `ExhibitorVisitorScan`, `ArchiveEdition` (+ `ArchiveMediaItem`, `ArchiveSessionTitle`, `ArchivePastSpeaker`), `Rating`, `Banner`, `Connection`, `Invitation`, `MeetingTable`, `HallAllocation`, `BusinessMeeting` (+

`BusinessMeetingParticipant`), `SpeakerMeetingRequest`, `SpeakerAvailabilityWindow`, `DelegationMeetingRequest`, `Contact`, `VisitorShareToken`, `SystemSetting`, `OrganizationProfile` (+ `OrganizationAboutItem`, `OrganizationDetail`), `AiPrompt`, `AiPromptHistory`, `AiInvocation`, `RowAudit`; and on `SIMF_Identity`: `SecondFactorToken`, `TotpRecoveryCode`, `DeviceKey`, `PasswordHistoryEntry`. The Identity-side `AccountCode` supersedes `EmailVerificationCode`, and `TotpSecret` is absorbed into the Identity token store.

Authoritative full schema. The complete per-column schema for all tables (approximately 100 across both databases) is defined by the `InitialModel`/`InitialCreate` and additive EF Core migrations under `src/Backend/SIMF.Infrastructure/Persistence/Migrations/{Identity,App}/` and the EF Core entity configurations (`SIMF.Infrastructure.Persistence.Configurations`). Exact column types, lengths and indexes are reviewed as code in those migrations and are not reproduced column-by-column here. Columns not shown in section 6.3.1 above are intentionally not reproduced here.

7. Solution Description

This section describes the technical architecture of the SIMF platform: the layered logical view, the concrete solution structure (technology stack, folder layouts, project dependencies), the patterns and practices applied throughout, and a categorised inventory of every third-party package. All facts are grounded in the current source tree and the deterministic project/package inventory read from the repository. Where a source marks something as reserved or not-yet-implemented, this section says so rather than inventing detail.

7.1 Logical View

SIMF is built as a modular monolith with Domain-Driven Design (DDD) layering. The backend is a single deployable API host organised into feature-aligned bounded contexts, with a strictly inward dependency rule: `Api → Infrastructure → Application → Domain`. This keeps the domain testable and isolates persistence concerns. Two web front-ends (Control Panel, Website) and one Flutter mobile app consume the API; the Control Panel and Website apply a Backend-for-Frontend (BFF) token-handling pattern so browsers never hold raw bearer tokens.

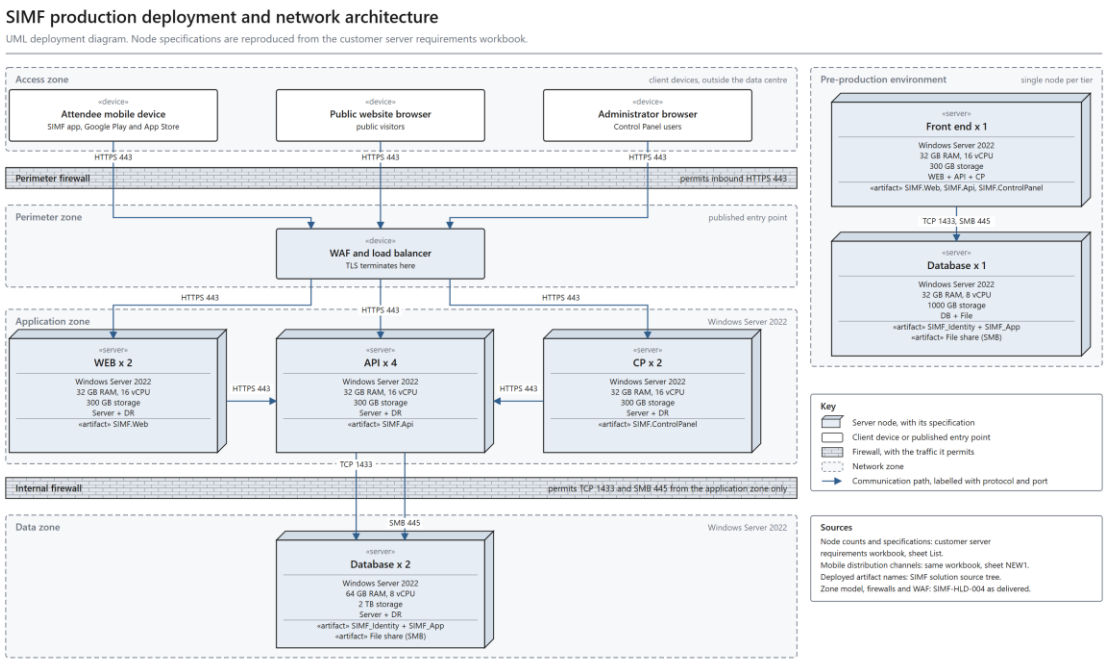

Figure 3. SIMF production deployment and network architecture. A UML deployment diagram: one node per hosting server, with its zone, its specification from the customer server requirements workbook, and the protocol and port of every path.

7.1.1 Backend layers

Layer	Project(s)	Responsibility
Domain	SIMF.Domain	Entities, aggregates, enums, domain rules; no framework dependencies
Application	SIMF.Application	Use cases, service abstractions; no ASP.NET/EF
Infrastructure	SIMF.Infrastructure	EF Core contexts, storage, e-mail, identity, JWT, audit interceptors
API (host)	SIMF.Api	FastEndpoints host, middleware, auth, policies, background workers
Real-time	REST polling	Live updates are polled over REST on a bounded interval. No server-push transport ships in this build.

7.1.2 Client and shared layers

- **Control Panel (BFF)**, Blazor Server, interactive-server render with prerendering disabled; cookie auth (`simf.cp.auth`); tokens held per-circuit and forwarded to the API by the typed client.
- **Website (BFF)**: Blazor SSR for public content with interactive islands for auth/account flows; cookie auth (`simf.web.auth`); binary auth (signed-in vs anonymous), no per-page permissions.

- **Flutter mobile**: layered into `lib/app` (shell/theme/localization/widgets), `lib/core` (env, networking, session, responsive, validation, utilities), and `lib/features` (per-feature screens), over two local packages `simf_data_pkg` (HTTP layer) and `simf_auth_pkg` (auth/session/device keys).
- **Shared libraries**: `SIMF.Common`, `SIMF.Contracts`, `SIMF.ApiClient`, `SIMF.Components`.

7.1.3 Cross-cutting concerns

These are implemented once and applied across all feature contexts:

Concern	Mechanism
Authentication	JWT bearer (HS256) default scheme + distinct StreamToken scheme
Authorisation	Named policies + dynamic permission-policy provider over `PermissionCatalog`
Validation	FluentValidation `Validator<TRequest>`; failures to HTTP 400 `VALIDATION_FAILED`
Correlation	`CorrelationIdMiddleware` reads/generates `X-Correlation-Id`, enriches logs
Error handling	`ErrorHandlingMiddleware` converts exceptions to `ApiResult.Fail`
Logging	Serilog; correlation-enriched
Auditing	`AuditStampingSaveChangesInterceptor` + `RowAuditingSaveChangesInterceptor` + explicit `OperationLog`
Soft-delete	`BaseAuditEntity.Deactivate()` flips `IsActive`; list queries filter `IsActive == true`
Rate limiting	Global per-IP + named policies (`auth`, `auth-email`, `ai-test`)

7.2 Solution Structure

7.2.1 Technology Stack

Versions below are the exact package/runtime versions read from the repository. Where the ground-truth inventory does not carry an explicit version for a technology, it is marked accordingly.

Layer	Technology	Version	Purpose
Runtime	.NET	10.0 (packages 10.0.x)	Backend runtime/framework
API framework	FastEndpoints	8.1.0	Endpoint host + routing
API docs	FastEndpoints.Swagger	8.1.0	OpenAPI/Swagger generation
Data access	EF Core (SqlServer)	10.0.8	ORM / migrations
Database	SQL Server	2022 Standard edition	Relational store (two DBs): SQL Server edition
Control Panel	Blazor Server	(ASP.NET Core 10)	Admin UI
Website	Blazor SSR	(ASP.NET Core 10)	Public site

Component library	SIMF.Components (Simf*) + Cropper.Blazor 1.5.1	10.0.7 (Components.Web)	CP/Web shared components + design tokens
Mobile	Flutter / Dart	Flutter 3.27.0 or later, Dart SDK 3.6.0 or later	Android / iOS app (pubspec `environment`)
Logging	Serilog.AspNetCore	10.0.0	Structured logging
E-mail	MailKit	4.16.0	SMTP send
Hosting	IIS (Windows Server 2022)	10.x	Reverse-proxied on-prem host

Note on the component library: the shared `Simf*` Blazor component library (`SIMF.Components`) and `theme.tokens.css` form the CP/Website UI foundation; the repository's package inventory does not pin a MudBlazor package (a single stray razor reference aside), so the UI foundation is recorded here as `SIMF.Components` plus `Cropper.Blazor` rather than MudBlazor.

7.2.2 Backend Folder Structure

Annotated tree of `src/` (backend + shared), from the repository project list and folder inventory:

```
src/
├─ Backend/
│  ├─ SIMF.Domain/ # entities, aggregates, enums, domain rules (no framework deps)
│  ├─ SIMF.Application/ # use cases + service abstractions (no ASP.NET/EF)
│  ├─ SIMF.Infrastructure/ # EF contexts, storage, email, identity, JWT, audit
│  └─ SIMF.Api/ # FastEndpoints host
└─ Shared/
   ├─ SIMF.Common/ # ApiResult<T>, PermissionCatalog, AppRoles, enums, GridQuery
   ├─ SIMF.Contracts/ # request/response DTOs
   ├─ SIMF.ApiClient/ # typed HTTP client for CP + Website
   └─ SIMF.Components/ # shared Simf* Blazor components + theme tokens
```

SIMF.Api internals (host-level folders):

```
SIMF.Api/
├─ App_Data/
├─ Authentication/ # JWT/StreamToken scheme setup
├─ Authorization/ # PermissionAuthorization (policy provider + handler)
├─ Endpoints/ # feature-grouped endpoint classes (see below)
├─ HostedServices/ # background workers (dormant sweep, reminders, etc.)
├─ Middleware/ # correlation, security headers, error handling, email-rate-key, swagger auth
├─ RateLimiting/
├─ RequestContext/ # HttpRequestContext (IRequestContext)
└─ logs/
```

`SIMF.Api/Endpoints/` is organised by feature folder (bounded-context aligned): Account, Admin, Ai, Archive, Assets, Attendance, Auth, BusinessMeetings, Contacts, Exhibitors, Feedback, Files, Gates,

Networking, News, Operations, Programme, Public, PublicRelations, Recommendations, Requests, Sessions, Sponsors, Staff, Statistics, Support.

SIMF.Infrastructure internals (feature/service folders): AccessControl, Ai, Archive, Assets, Attendance, Auditing, BusinessMeetings, Cms, Common, Configuration, Contacts, Delegations, Email, Excel, Exhibition, Exhibitors, Faq, Feedback, Files, Identity, IdentityAccess, Logs, Media, MeetingRequests, MyArea, Networking, Operations, Organisations, Persistence, Programme, PublicRelations, Recommendations, Regions, Requests, SeatReservations, Seeding, SessionQuestions, Sponsors, Statistics, Support, Venue. (Full list; not summarised.)

7.2.3 Frontend Folder Structure

Flutter app (`src/Mobile/simf_app`):

```
lib/
 ├– app/
 |  ├– localization/ # bilingual (Arabic/English) resources
 |  ├– theme/ # navy-always theme
 |  └– widgets/ # global app-bar invariant + shared widgets
 ├– core/
 |  ├– env/ # environment / API base
 |  ├– errors/
 |  ├– net/ # networking glue
 |  ├– organization_profile/
 |  ├– responsive/ # flexible-width helpers
 |  ├– session/
 |  ├– sharing/
 |  ├– site_settings/
 |  ├– startup/ # cold-start auth restore / splash gate
 |  ├– utils/
 |  ├– validation/
 |  └– widgets/
 └– features/ # ~34 feature folders (per-screen)
   ├– about, accessibility, account, ai_summary, archive, badge, booths,
   ├– chatbot, contact_us, contacts, content, delegations, exhibition,
   ├– exhibitor, faq, feedback, forum_guide, gallery, gates, guest, home,
   ├– live, media_partners, meet, meetings, moderation, more, myarea, news,
   ├– notifications, onboarding, questions, registration, requests, sessions,
   └– speakers, splash, sponsors, staff, venuemap
```

Local packages (`src/Mobile/packages/`, tracked): `simf_data_pkg` (single HTTP layer: `SimfApiClient`, `ApiResult`, `ApiFailure`, secure/prefs storage) and `simf_auth_pkg` (auth controller/state, session, AppRole, biometric ES256 device keys).

Blazor CP / Website (folder-level detail below is representative, not an exhaustive tree; the authoritative layout is the projects `src/ControlPanel/SIMF.ControlPanel` and `src/Website/SIMF.Web`):

```
src/
 ├─ ControlPanel/SIMF.ControlPanel/
 |  ├─ Endpoints/ # AuthEndpoints.cs (BFF credential/2FA + cookie)
 |  ├─ Authorization/ # PermissionAuthorization.cs (policy provider + handler)
 |  ├─ CpNavigation.cs # ~12 nav groups over /admin/* routes
 |  └─ (pages using SimfDataGrid + CRUD framework)
 └─ Website/SIMF.Web/
    └─ (SSR public pages + interactive auth/account islands)
```

7.2.4 Project Dependencies

Project-reference graph. The dependency rule is strictly inward: outer layers reference inner layers, never the reverse.

Project	References	Direction note
SIMF.Domain	SIMF.Common	Innermost; no framework deps
SIMF.Application	SIMF.Domain, SIMF.Common, SIMF.Contracts	Inward to Domain
SIMF.Infrastructure	SIMF.Domain, SIMF.Application	Inward to Application/Domain
SIMF.Api	SIMF.Application, SIMF.Infrastructure, SIMF.Common, SIMF.Contracts	Host; composes all
SIMF.Common	-	Shared kernel; no references
SIMF.Contracts	SIMF.Common	DTOs
SIMF.ApiClient	SIMF.Common, SIMF.Contracts	Typed client for CP + Website
SIMF.Components	Microsoft.AspNetCore.Components.Web	Shared Blazor components + theme
SIMF.ControlPanel	SIMF.ApiClient, SIMF.Common, SIMF.Contracts, SIMF.Components	Blazor Server admin BFF
SIMF.Web	SIMF.ApiClient, SIMF.Common, SIMF.Contracts, SIMF.Components	Blazor SSR public site + BFF

The dependency rule is strictly inward, Domain to Application to Infrastructure to Api, which keeps the domain testable and the persistence concerns isolated. The Control Panel and Website reference only the shared client/contract/component projects (`SIMF.ApiClient`, `SIMF.Common`, `SIMF.Contracts`, `SIMF.Components`), never the backend directly, so they call the API server-to-server through the typed client.

7.2.5 Patterns & Practices

Applied consistently across the backend:

- **FastEndpoints endpoint pattern**: each endpoint is a class with `Configure()` (route, verbs, policies/anonymous, tags, rate limiting, summary) and `HandleAsync()`; app routes under `/api/v1/app/*`, admin under `/api/v1/admin/*`.
- **`ApiResult<T>` envelope**: every response is `ApiResult<T>` with `Success`, `Data`, `Error` (`ApiError` carrying `Code`, `Message`, `MessageArabic`, and `ApiErrorDetail` list) and `Meta` for pagination/counts; `ApiCallResult<T>` pairs an HTTP status with the envelope.
- **FluentValidation triple-lock**: validation lengths align across FluentValidation, EF `HasMaxLength`, and the UI `MaxLength`; failures convert to HTTP 400 `VALIDATION_FAILED` with bilingual field detail.
- **Permission catalogue**: `PermissionCatalog` (`src/Shared/SIMF.Common/PermissionCatalog.cs`) is the single source of truth: `Wildcard = "*"`, `ClaimType = "perm"`, `PolicyPrefix = "perm:"`, nested `Page.Action` codes; reused by both API and Control Panel, with build-failing tests for ungated pages/endpoints.
- **Repository/service abstractions**: use-case services live in `SIMF.Application` behind interfaces (e.g. `IJwtTokenService`, `IRequestContext`, storage interfaces `IAvatarStorage`/`ISessionRecordingStorage`/etc.); implementations in `SIMF.Infrastructure`.
- **Soft-delete**: `BaseAuditEntity.Deactivate()` flips `IsActive`; list queries filter `IsActive == true`.
- **Audit interceptors**: `AuditStampingSaveChangesInterceptor` (created/updated stamps) and `RowAuditingSaveChangesInterceptor` (row-level change log) run on the App context; the Identity context runs the row-audit interceptor; both audit tables are append-only at the application level.
- **Options pattern**: typed config sections bound via the Options pattern; production overrides/secrets via `SIMF_`-prefixed environment variables (double-underscore nesting, e.g. `SIMF_Jwt__SigningKey`); boot guards refuse to start when required secrets are missing.
- **Guard clauses / deterministic failure**: `DataValidationException` (extends `ApiException`) for validation; error middleware maps exceptions to the correct status.
- **Pagination**: app GETs use `?page=&pageSize=&sort=&search=`; admin grids POST a `GridQuery` body returning `GridPage<T>`; default page size 20, grid cap 200.

Naming / error-handling / logging / security conventions: feature-folder organisation mirrored across `Endpoints/`, `Application/`, `Infrastructure/`; bilingual `Name`/`NameArabic` fields; UTC timestamps; correlation-ID propagation through logs and audit rows; CORS explicit allow-list (never wildcard); Swagger gated behind Basic auth plus flag in production with a boot guard.

7.2.6 Packages Inventory

The backend tables below list the **exact** NuGet packages and versions read from the repository (deduped across projects). Only packages present in the ground-truth inventory are listed.

Framework / Runtime

Package	Version	Purpose
Microsoft.AspNetCore.Components.Web	10.0.7	Blazor component primitives
Microsoft.Extensions.Configuration.Abstractions	10.0.8	Config abstractions
Microsoft.Extensions.Hosting.Abstractions	10.0.8	Hosting abstractions

Microsoft.Extensions.Options.ConfigurationExtensions	10.0.8	Options-pattern binding
Microsoft.Extensions.Logging.Abstractions	10.0.8	Logging abstractions

Data Access

Package	Version	Purpose
Microsoft.EntityFrameworkCore.SqlServer	10.0.8	EF Core SQL Server provider
Microsoft.EntityFrameworkCore.Design	10.0.8	EF migrations / design-time

Auth & Security

Package	Version	Purpose
Microsoft.AspNetCore.Authentication.JwtBearer	10.0.8	JWT bearer authentication
Microsoft.AspNetCore.Identity.EntityFrameworkCore	10.0.8	ASP.NET Identity stores
Microsoft.Extensions.Identity.Stores	10.0.8	Identity store primitives
System.IdentityModel.Tokens.Jwt	8.18.0	JWT token creation/validation
Otp.NET	1.4.1	TOTP second factor

API & Serialization

Package	Version	Purpose
FastEndpoints	8.1.0	Endpoint framework
FastEndpoints.Swagger	8.1.0	OpenAPI/Swagger docs

Logging

Package	Version	Purpose
Serilog.AspNetCore	10.0.0	Structured logging integration
Serilog.Sinks.File	7.0.0	File log sink

Imaging / QR / ML

Package	Version	Purpose
QRCoder	1.6.0	QR code generation
Cropper.Blazor	1.5.1	Image cropping (CP UI)
FaceAiSharp.Bundle	0.6.35	Face processing (identity/liveness)
Microsoft.ML.OnnxRuntime	1.26.0	ONNX model inference

E-mail / Office / Other Infrastructure

Package	Version	Purpose
MailKit	4.16.0	SMTP e-mail send
ClosedXML	0.104.2	Excel import/export

Flutter packages (from `pubspec`; caret constraints as recorded):

Package	Version	Purpose
flutter_riverpod	^2.6.1	State management
go_router	^14.6.0	Routing / shell navigation
intl	^0.19.0	i18n / formatting

flutter_localizations	(SDK)	Localization delegates
local_auth	^2.3.0	Biometric auth
image_picker	^1.1.2	Image selection
google_mlkit_face_detection	^0.13.1	Face detection (liveness)
camera	^0.11.0+2	Camera capture
share_plus	^10.0.0	Native share
add_2_calendar	^3.0.1	Add session to calendar
qr_flutter	^4.1.0	QR rendering (badge)
video_player	^2.9.1	Video playback (HLS/MP4 fallback)
youtube_player_iframe	^6.0.2	Live/sign-language YouTube feed
url_launcher	^6.3.0	External links
flutter_zxing	^2.3.0	QR/barcode scanning
flutter_svg	^2.0.10	SVG rendering
package_info_plus	^8.3.0	App version info
pub_semver	^2.1.4	Version comparison
pretty_dio_logger	^1.4.0	Dev/test HTTP logging
simf_data_pkg	(local)	HTTP/data layer package
simf_auth_pkg	(local)	Auth/session package